

Indeksy, optymalizator

Lab 2

Imiona i nazwiska: Natalia Bratek i Jakub Karczewski

Celem ćwiczenia jest zapoznanie się z planami wykonania zapytań (execution plans), oraz z budową i możliwością wykorzystaniem indeksów (kontynuacja poprzedniego ćwiczenia)

Swoje odpowiedzi wpisuj w miejsca oznaczone jako:

Wyniki:

-- ...

Ważne/wymagane są komentarze.

Zamieść kod rozwiązania oraz zrzuty ekranu pokazujące wyniki, (dołącz kod rozwiązania w formie tekstowej/źródłowej)

Zwróć uwagę na formatowanie kodu

Oprogramowanie - co jest potrzebne?

Do wykonania ćwiczenia potrzebne jest następujące oprogramowanie

- MS SQL Server
- SSMS - SQL Server Management Studio
 - ewentualnie inne narzędzie umożliwiające komunikację z MS SQL Server i analizę planów zapytań

Oprogramowanie dostępne jest na przygotowanej maszynie wirtualnej

Przygotowanie

Uruchom Microsoft SQL Managment Studio.

Stwórz swoją bazę danych o nazwie lab2.

```
create database lab2
go

use lab2
go
```

Warto przełączyć bazę w tryb simple

```
alter database lab2  
set recovery simple;
```

Zadanie 1 - indeksy

Wykonaj poniższy skrypt, aby przygotować dane:

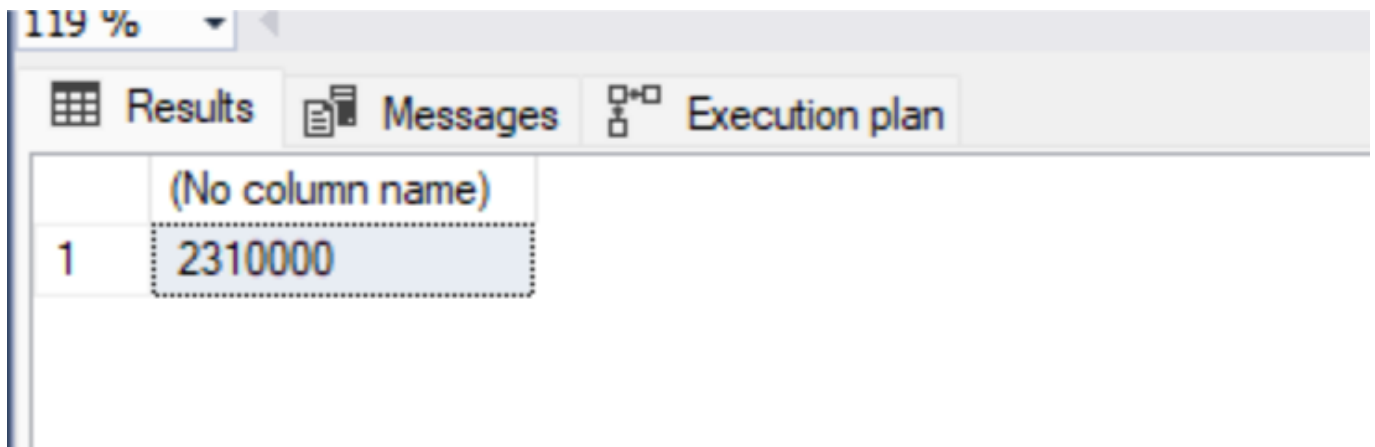
```
select * into product_history
from northwind3.dbo.product_history

select * into categories
from northwind3.dbo.categories

create clustered index categ_clust_idx
on categories(categoryid)
```

sprawdź liczbę wierszy w tabeli

```
select count(*) from product_history
```



	(No column name)
1	2310000

Sprawdź jakie indeksy istnieją dla tej tabeli

```
exec sp_helpindex 'dbo.product_history'
```

```
Select
    i.name as index_name,
    i.type_desc,
    i.is_unique,
    c.name as column_name,
    ic.key_ordinal,
    ic.is_included_column
from sys.indexes i
```

```
join sys.index_columns ic
  on i.object_id = ic.object_id
  and i.index_id = ic.index_id
join sys.columns c
  on ic.object_id = c.object_id
  and ic.column_id = c.column_id
where i.object_id = object_id('dbo.product_history')
order by i.name, ic.key_ordinal;
```

19 %

Results Messages Execution plan

index_name	type_desc	is_unique	column_name	key_ordinal	is_included_column
------------	-----------	-----------	-------------	-------------	--------------------

włącz statystyki IO i TIME

```
SET STATISTICS IO ON

SET STATISTICS TIME ON;
```

podczas analiz sprawdzaj jak zachowują się zapytania, zwróć uwagę na

- plan
- koszt
- czas (ewentualnie, jeśli coś da się zaobserwować)
- liczbę odczytywanych stron !!!!

porównaj zapytania

a)

```
select count(*) from product_history
where id = 1000000
```

119 %

Results Messages Execution plan

	(No column name)
1	1

Results Messages Execution plan

CPU time = 0 ms, elapsed time = 4 ms.
SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

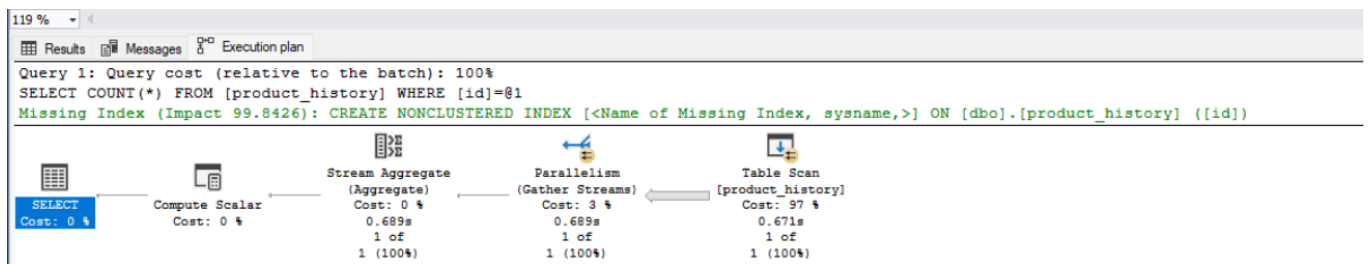
(1 row affected)
Table 'product_history'. Scan count 5, logical reads 25265, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0.

(1 row affected)

SQL Server Execution Times:
CPU time = 313 ms, elapsed time = 142 ms.
SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

SQL Server Execution Times:
CPU time = 0 ms, elapsed time = 0 ms.

Completion time: 2026-05-08T10:40:35.5611614+02:00



Query 1: Query cost (relative to the batch): 100%

```
SELECT COUNT(*) FROM [product_history] WHERE [id]=@1
```

Missing Index (Impact 99.8426): CREATE NONCLUSTERED INDEX

SELECT	
Cached plan size	24 KB
Estimated Operator Cost	0 (0%)
Degree of Parallelism	4
Estimated Subtree Cost	20,5707
Memory Grant	72 KB
Estimated Number of Rows for All Executions	0
Estimated Number of Rows Per Execution	1
Statement	
SELECT COUNT(*) FROM [product_history] WHERE [id]=@1	

```
select count(*) from product_history
where id between 999000 and 1000000
```

119 %

Results	
(No column name)	
1	1311001

119 %

Results Messages Execution plan

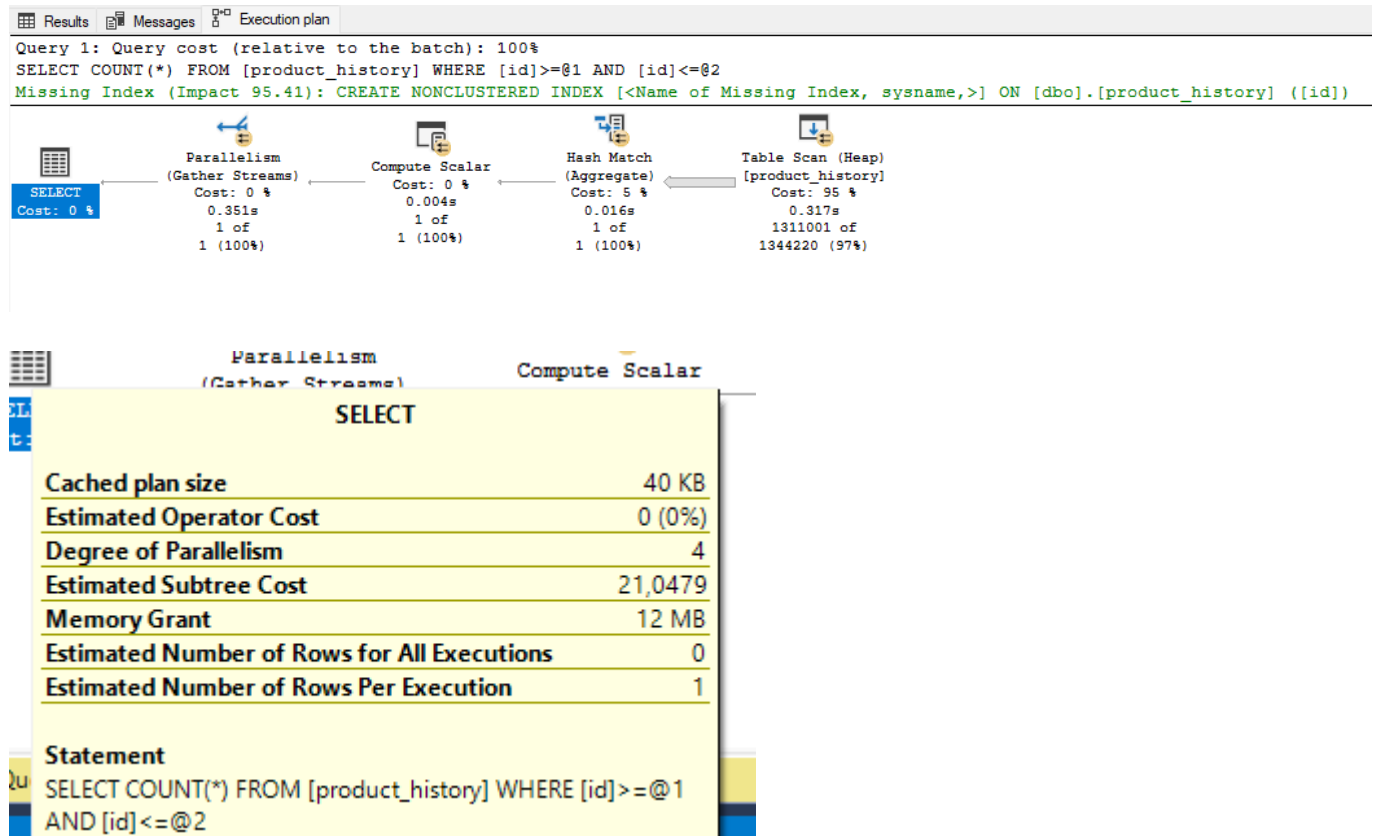
SQL Server Execution Times:
 CPU time = 0 ms, elapsed time = 0 ms.
 SQL Server parse and compile time:
 CPU time = 0 ms, elapsed time = 74 ms.
 SQL Server parse and compile time:
 CPU time = 0 ms, elapsed time = 0 ms.

(1 row affected)

Table 'product_history'. Scan count 5, logical reads 25265, physical reads 0, page server reads 0, read-ahead reads 25272, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, read-ahead reads 0, page server read-ahead reads 0.

(1 row affected)

SQL Server Execution Times:
 CPU time = 548 ms, elapsed time = 640 ms.
 SQL Server parse and compile time:
 CPU time = 0 ms, elapsed time = 0 ms.



Zapytanie	Plan / Operator	Logical reads	Estimated Cost	CPU / Elapsed
count(*) where id = 1000000	Table Scan + Stream Aggregate	25 265	20,57	313 / 142 ms
count(*) where id between 999000 and 10000000	Table Scan + Hash Match	25 265	21,05	548 / 640 ms

Oba zapytania wykonują pełen Table Scan i odczytują tę samą liczbę stron. Przeglądana jest cała tabela, ponieważ SQL Server nie wie gdzie znajdują się szukane wiersze. Pokazuje się Missing Index Hint (Impact 99,84%), SQL sugeruje utworzenie indeksu na kolumnie id.

b)

```
select * from product_history
where id = 1000000
```

119 %

	id	productid	productname	supplierid	categoryid	quantityperunit	unitprice	quantity	value	date
1	1000000	1	Chai	1	1	10 boxes x 20 bags	14.47	10	144.70	1975-07-24

Results Messages Execution plan

SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

SQL Server Execution Times:
CPU time = 0 ms, elapsed time = 0 ms.

SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 2 ms.

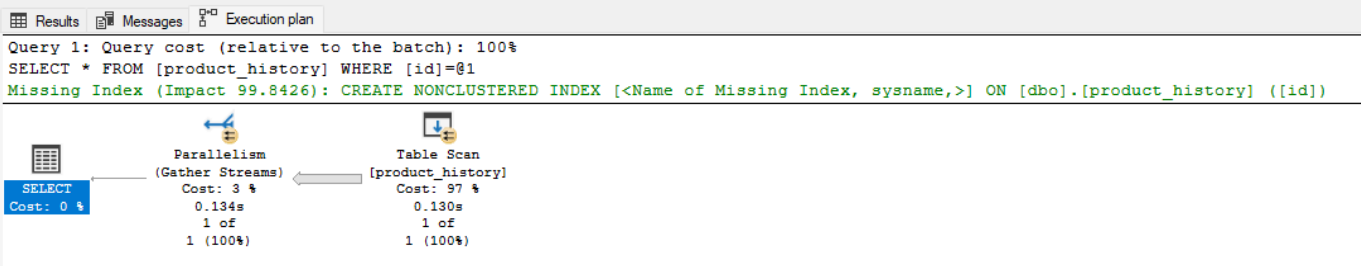
SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

(1 row affected)
Table 'product_history'. Scan count 5, logical reads 25265, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0

(1 row affected)

SQL Server Execution Times:
CPU time = 249 ms, elapsed time = 193 ms.

SQL Server parse and compile time:



Parallelism Table Scan

SELECT

Cached plan size	32 KB
Estimated Operator Cost	0 (0%)
Degree of Parallelism	4
Estimated Subtree Cost	20,5707
Memory Grant	72 KB
Estimated Number of Rows for All Executions	0
Estimated Number of Rows Per Execution	1,0195

Statement

SELECT * FROM [product_history] WHERE [id]=@1

```
select * from product_history
where id between 999000 and 1000000
```


119 %										
Results Messages										
	id	productid	productname	supplierid	categoryid	quantityperunit	unitprice	quantity	value	date
1	1008466	74	Longlife Tofu	4	7	5 kg pkg.	15.95	18	287.10	1975-11-10
2	1008467	75	Rhönbräu Klosterbier	12	1	24 - 0.5 l bottles	14.61	18	262.98	1975-11-10
3	1008468	76	Lakkalikööri	23	1	500 ml	20.71	18	372.78	1975-11-10
4	1008469	77	Original Frankfurter grüne Soße	12	2	12 boxes	17.73	18	319.14	1975-11-10
5	1008470	1	Chai	1	1	10 boxes x 20 bags	14.33	10	143.30	1975-11-11
6	1008471	2	Chang	1	1	24 - 12 oz bottles	14.58	10	145.80	1975-11-11
7	1008472	3	Aniseed Syrup	1	2	12 - 550 ml bottles	12.41	10	124.10	1975-11-11
8	1008473	4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	15.30	10	153.00	1975-11-11
9	1008474	5	Chef Anton's Gumbo Mix	2	2	36 boxes	15.14	10	151.40	1975-11-11
10	1008475	6	Grandma's Boysenberry Spread	3	2	12 - 8 oz jars	16.02	10	160.20	1975-11-11
11	1008476	7	Uncle Bob's Organic Dried P...	3	7	12 - 1 lb pkgs.	17.22	10	172.20	1975-11-11
12	1008477	8	Northwoods Cranberry Sauce	3	2	12 - 12 oz jars	19.63	10	196.30	1975-11-11
13	1008478	9	Mishi Kobe Niku	4	6	18 - 500 g pkgs.	33.36	10	333.60	1975-11-11
14	1008479	10	Ikura	4	8	12 - 200 ml jars	17.46	10	174.60	1975-11-11
15	1008480	11	Queso Cabrales	5	4	1 kg pkg.	15.06	10	150.60	1975-11-11
16	1008481	12	Queso Manchego La Pastora	5	4	10 - 500 g pkgs.	19.15	10	191.50	1975-11-11

Results Messages Execution plan		
SQL Server Execution Times: CPU time = 0 ms, elapsed time = 0 ms.		
SQL Server parse and compile time: CPU time = 0 ms, elapsed time = 15 ms.		
SQL Server parse and compile time: CPU time = 0 ms, elapsed time = 0 ms.		
(1311001 rows affected)		
Table 'product_history'. Scan count 1, logical reads 25265, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0		
(1 row affected)		
SQL Server Execution Times: CPU time = 4235 ms, elapsed time = 67808 ms.		
SQL Server parse and compile time: CPU time = 0 ms, elapsed time = 0 ms.		
SQL Server Execution Times: CPU time = 0 ms, elapsed time = 0 ms.		
Completion time: 2026-05-08T10:29:40.0798665+02:00		

Results Messages Execution plan		
Query 1: Query cost (relative to the batch): 100%		
SELECT * FROM [product_history] WHERE [id]>=@1 AND [id]<=@2		
Missing Index (Impact 69.4934): CREATE NONCLUSTERED INDEX [<Name of Missing Index, sysname,>] ON [dbo].[product_history] ([id]) INCLUDE ([...]		
SELECT * FROM [product_history] WHERE [id]>=@1 AND [id]<=@2		
Table Scan [product_history] Cost: 100 % 7.519s 1311001 of 1344220 (97%)		

[product_history]	
SELECT	
Cost: 100 %	
SELECT	
Cached plan size	24 KB
Estimated Operator Cost	0 (0%)
Degree of Parallelism	1
Estimated Subtree Cost	21,2584
Estimated Number of Rows for All Executions	0
Estimated Number of Rows Per Execution	1344220
Statement	
SELECT * FROM [product_history] WHERE [id]>=@1 AND [id]<=@2	

Zapytanie	Plan	Logical reads	Estimated Cost	CPU / Elapsed
<code>select * where id = 1000000</code>	Table Scan + Parallelism	25 265	20,57	249 / 193 ms
<code>select * where id between 999000 and 10000000</code>	Table Scan	25 265	21,26	4235 / 67 808 ms

Logical reads pozostają identyczne, bo bez indeksu zawsze trzeba przeczytać całą tabelę. Missing Index Hint w pierwszym zapytaniu sugeruje zwykły indeks na id (Impact 99,84%), a w drugim indeks z INCLUDE wszystkich kolumn (Impact 69,49%), bo zapytanie potrzebuje wszystkich pól. W obu zapytaniach występuje Table Scan. Czas wykonania znacząco się różni. Pierwsze zapytanie 193 ms, drugie 67,8 s.

c)

sprawdź jak zachowują się zapytania z pkt a) i b) jeśli dla kolumny `id` stworzysz indeks

- klastrowy
- nieklastrowy

```
create clustered index product_history_clust_idx
on product_history(id)

drop index product_history_clust_idx on product_history

create index product_history_idx
on product_history(id)

drop index product_history_idx on product_history
```

indeks klastrowy

pkt a)

- pierwsze zapytanie

```
119 %
Results Messages Execution plan
CPU time = 0 ms, elapsed time = 1 ms.
SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

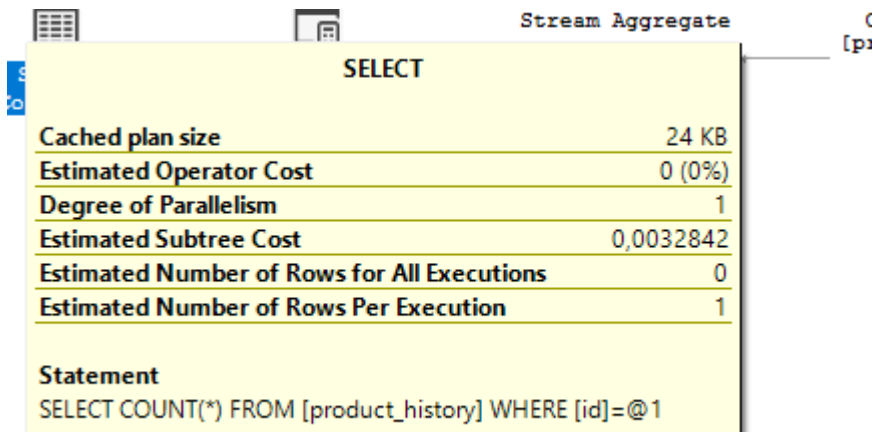
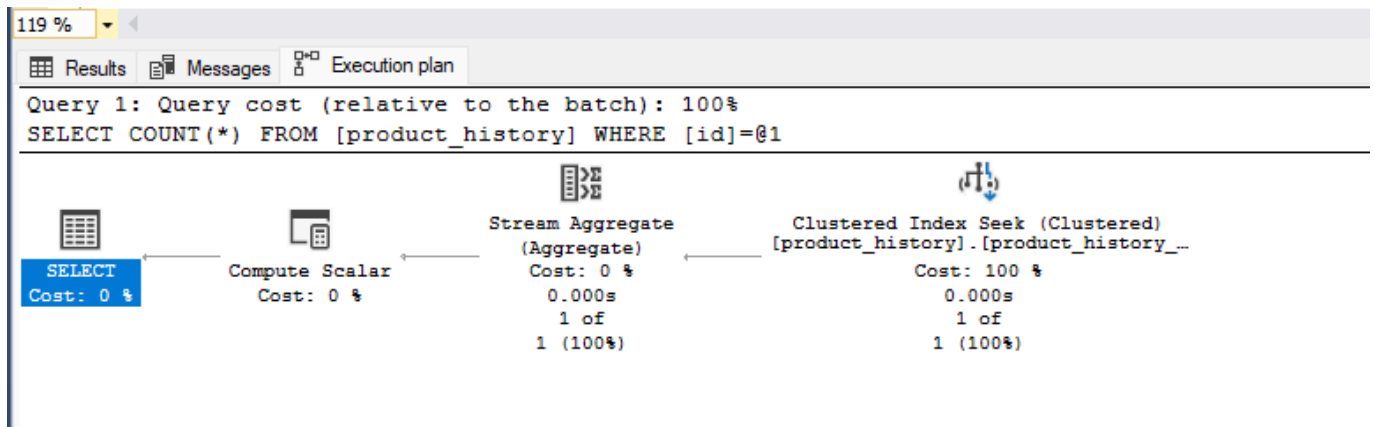
(1 row affected)
Table 'product_history'. Scan count 1, logical reads 3, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0.

(1 row affected)

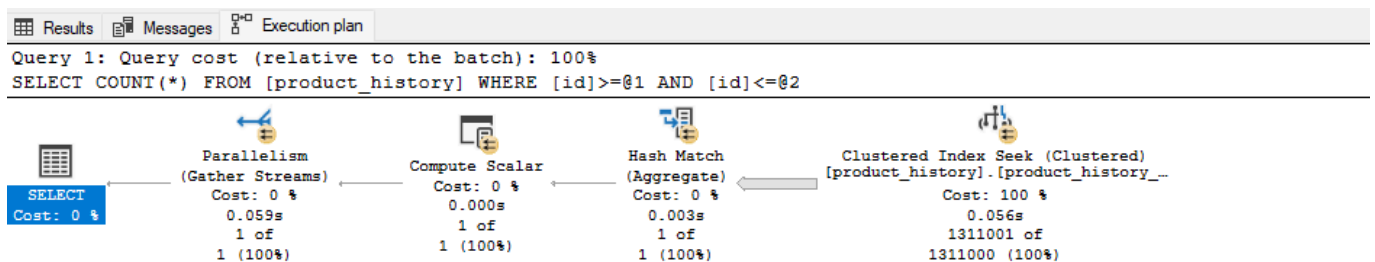
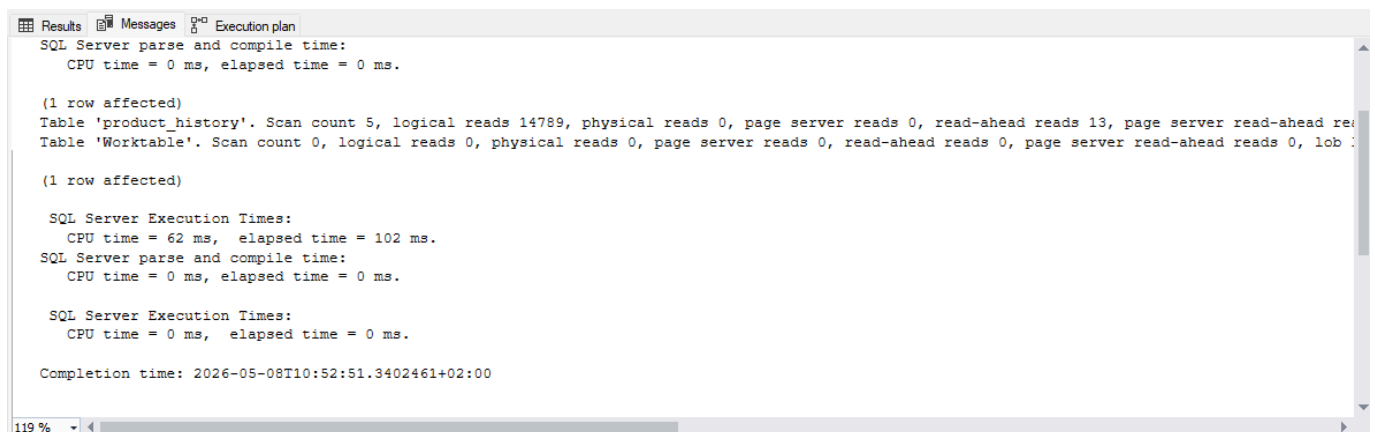
SQL Server Execution Times:
CPU time = 0 ms, elapsed time = 39 ms.
SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

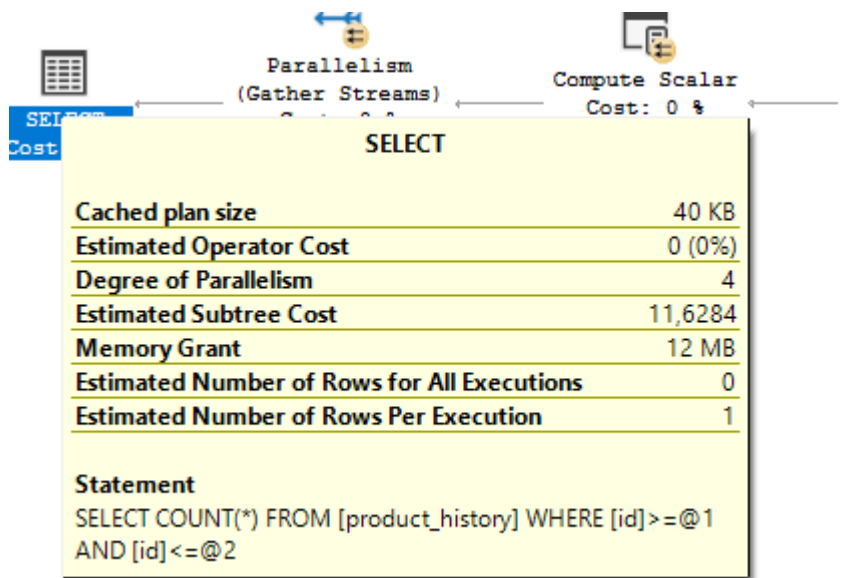
SQL Server Execution Times:
CPU time = 0 ms, elapsed time = 0 ms.

Completion time: 2026-05-08T10:46:28.5616650+02:00
```



- drugie zapytanie





pkt b)

- pierwsze zapytanie

```
Results Messages Execution plan
CPU time = 0 ms, elapsed time = 6 ms.
SQL Server parse and compile time:
  CPU time = 0 ms, elapsed time = 0 ms.

(1 row affected)
Table 'product_history'. Scan count 1, logical reads 3, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0.

(1 row affected)

SQL Server Execution Times:
  CPU time = 0 ms, elapsed time = 44 ms.
SQL Server parse and compile time:
  CPU time = 0 ms, elapsed time = 0 ms.

SQL Server Execution Times:
  CPU time = 0 ms, elapsed time = 0 ms.

Completion time: 2026-05-08T10:59:32.4538563+02:00
```

119 %

Results Messages Execution plan

Query 1: Query cost (relative to the batch): 100%

SELECT * FROM [product_history] WHERE [id]=@1

Clustered Index Seek (Clustered)
[product_history].[product_history_...]
Cost: 100 %
0.000s
1 of
1 (100%)

SELECT
Cost: 0 %

Clustered Index Seek (Clustered)
[product_history].[product_history_...]
Cost: 100 %

SELECT

Cached plan size	24 KB
Estimated Operator Cost	0 (0%)
Degree of Parallelism	1
Estimated Subtree Cost	0,0032831
Estimated Number of Rows for All Executions	0
Estimated Number of Rows Per Execution	1

Statement
SELECT * FROM [product_history] WHERE [id]=@1

- drugie zapytanie

119 %

Results Messages

	id	productid	productname	supplierid	categoryid	quantityperunit	unitprice	quantity	value	date
1	999000	2	Chang	1	1	24 - 12 oz bottles	27.30	10	273.00	1975-07-11
2	999001	3	Aniseed Syrup	1	2	12 - 550 ml bottles	19.11	10	191.10	1975-07-11
3	999002	4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	30.04	10	300.40	1975-07-11
4	999003	5	Chef Anton's Gumbo Mix	2	2	36 boxes	29.44	10	294.40	1975-07-11
5	999004	6	Grandma's Boysenberry Spread	3	2	12 - 8 oz jars	32.77	10	327.70	1975-07-11
6	999005	7	Uncle Bob's Organic Dried Pears	3	7	12 - 1 lb pkgs.	37.32	10	373.20	1975-07-11
7	999006	8	Northwoods Cranberry Sauce	3	2	12 - 12 oz jars	46.43	10	464.30	1975-07-11
8	999007	9	Mishi Kobe Niku	4	6	18 - 500 g pkgs.	98.34	10	983.40	1975-07-11
9	999008	10	Ikura	4	8	12 - 200 ml jars	38.23	10	382.30	1975-07-11
10	999009	11	Queso Cabrales	5	4	1 kg pkg.	29.13	10	291.30	1975-07-11
11	999010	12	Queso Manchego La Pastora	5	4	10 - 500 g pkgs.	44.61	10	446.10	1975-07-11
12	999011	13	Konbu	6	8	2 kg box	15.46	10	154.60	1975-07-11
13	999012	14	Tofu	6	7	40 - 100 g pkgs.	31.18	10	311.80	1975-07-11
14	999013	15	Genen Shouyu	6	2	24 - 250 ml bottles	24.12	10	241.20	1975-07-11
15	999014	16	Pavlova	7	3	32 - 500 g boxes	25.89	10	258.90	1975-07-11

Results Messages Execution plan

SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

SQL Server Execution Times:
CPU time = 0 ms, elapsed time = 0 ms.

SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

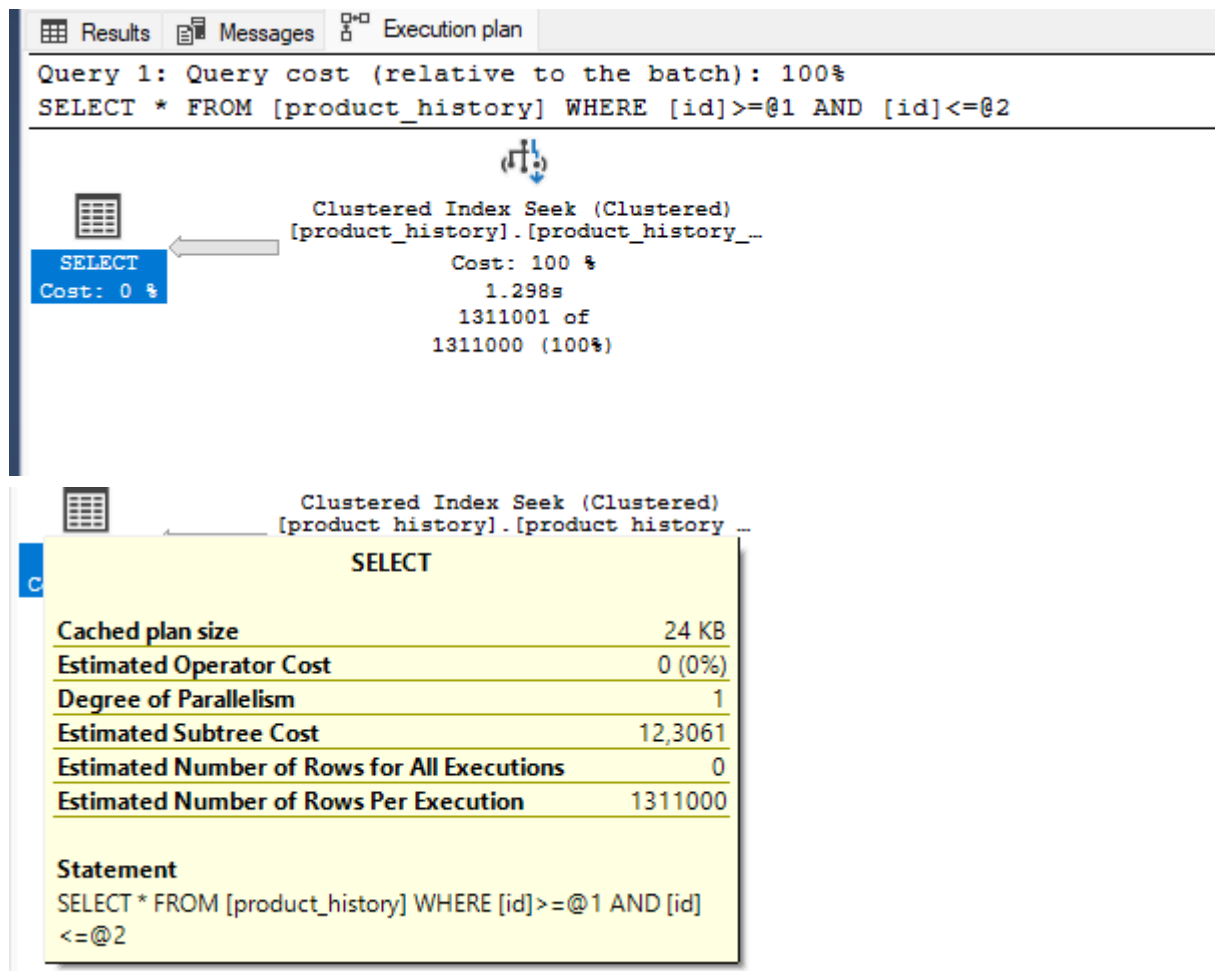
(1311001 rows affected)
Table 'product_history'. Scan count 1, logical reads 14691, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0.

(1 row affected)

SQL Server Execution Times:
CPU time = 1829 ms, elapsed time = 23299 ms.

SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

SQL Server Execution Times:
CPU time = 0 ms, elapsed time = 0 ms.



Zapytanie	Plan	Logical reads	Estimated Cost	CPU / Elapsed
a1) count(*) where id = 1000000	Clustered Index Seek	3	0,003	0 / 39 ms
a2) count(*) where id between 999000 and 1000000	Clustered Index Seek + Hash Match (parallel)	14 789	11,63	62 / 102 ms
b1) select * where id = 1000000	Clustered Index Seek	3	0,003	0 / 0 ms

Zapytanie	Plan	Logical reads	Estimated Cost	CPU / Elapsed
b2) <code>select * where id between 999000 and 10000000</code>	Clustered Index Seek (Range Scan)	14 691	12,31	1829 / 23 299 ms

Indeks klastrowy dla zapytań `where id = 1000000` zredukował logical reads z 25 265 do 3, dla zapytań `where id between 999000 and 10000000` zredukował do 14 789 (i 14 691). Czas wykonania zapytań `where id = 1000000` spadł niemal do zera (39 ms / 0 ms). Dla zapytań `where id between 999000 and 10000000` z `select *` elapsed time spadł do 23,3 s, a z `count(*)` do 102 ms.

indeks nieklastrowy

pkt a)

- pierwsze zapytanie

```

Results Messages Execution plan
CPU time = 0 ms, elapsed time = 0 ms.

(1 row affected)
Table 'product_history'. Scan count 1, logical reads 3, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0.

(1 row affected)

SQL Server Execution Times:
    CPU time = 0 ms,  elapsed time = 34 ms.
SQL Server parse and compile time:
    CPU time = 0 ms,  elapsed time = 0 ms.

SQL Server Execution Times:
    CPU time = 0 ms,  elapsed time = 0 ms.

Completion time: 2026-05-08T11:13:16.3088831+02:00

```

Query 1: Query cost (relative to the batch): 100%

`SELECT COUNT(*) FROM [product_history] WHERE [id]=@1`

```

graph LR
    A[SELECT  
Cost: 0 %] --> B[Compute Scalar  
Cost: 0 %]
    B --> C[Stream Aggregate  
(Aggregate)  
Cost: 0 %  
0.000s  
1 of 1 (100%)]
    C --> D[Index Seek (NonClustered)  
[product_history].[product_history_...]  
Cost: 100 %  
0.000s  
1 of 1 (100%)]

```

SELECT	
Cached plan size	24 KB
Estimated Operator Cost	0 (0%)
Degree of Parallelism	1
Estimated Subtree Cost	0,0032842
Estimated Number of Rows for All Executions	0
Estimated Number of Rows Per Execution	1
Statement	
<code>SELECT COUNT(*) FROM [product_history] WHERE [id]=@1</code>	

- drugie zapytanie

Results Messages Execution plan

SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 1 ms.

SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

(1 row affected)
Table 'product_history'. Scan count 1, logical reads 2934, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead read

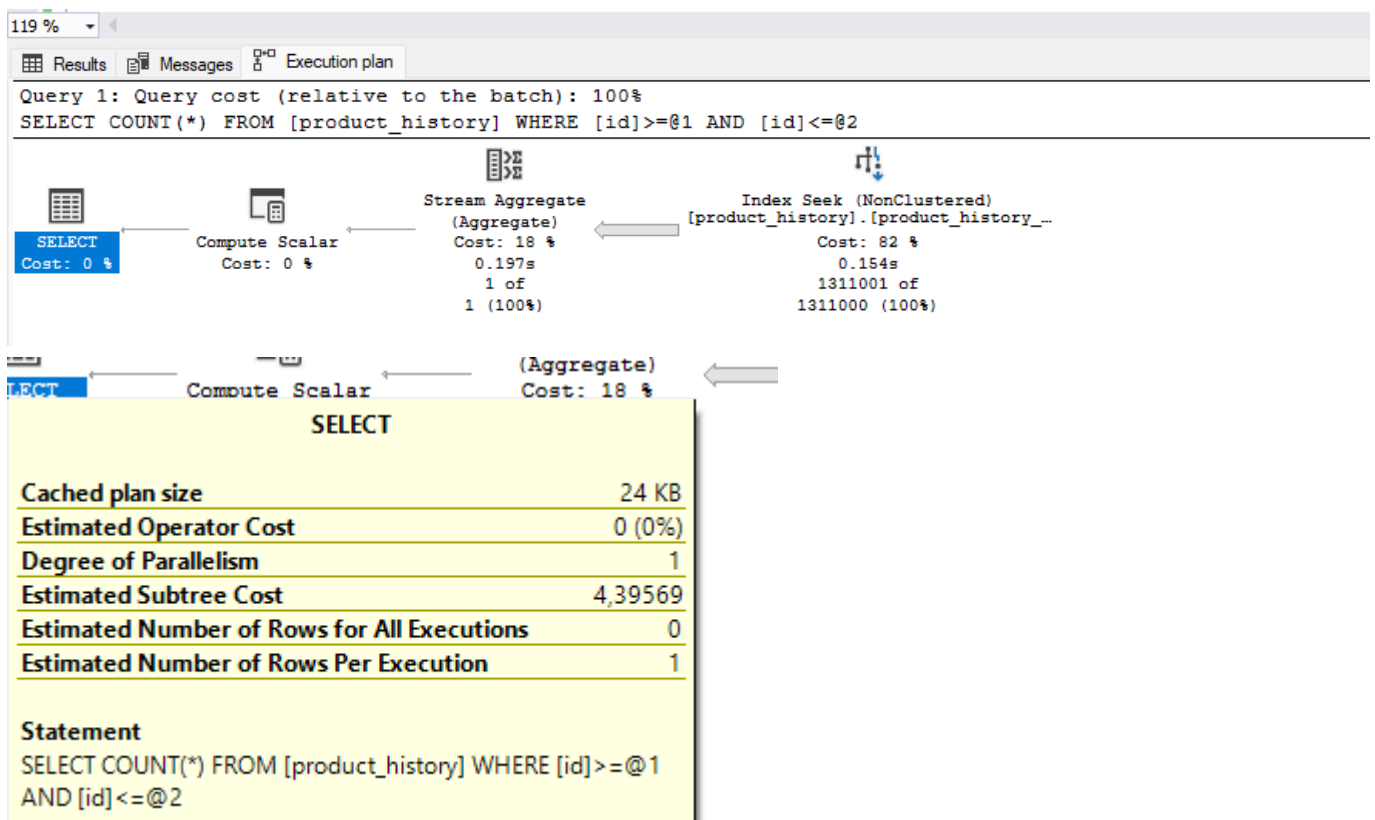
(1 row affected)

SQL Server Execution Times:
CPU time = 171 ms, elapsed time = 224 ms.

SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

SQL Server Execution Times:
CPU time = 0 ms, elapsed time = 0 ms.

Completion time: 2026-05-08T11:18:12.1429091+02:00



pkt b)

- pierwsze zapytanie

119 %

Results Messages Execution plan

SQL Server parse and compile time:
CPU time = 4 ms, elapsed time = 4 ms.

SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

(1 row affected)
Table 'product_history'. Scan count 1, logical reads 4, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0,

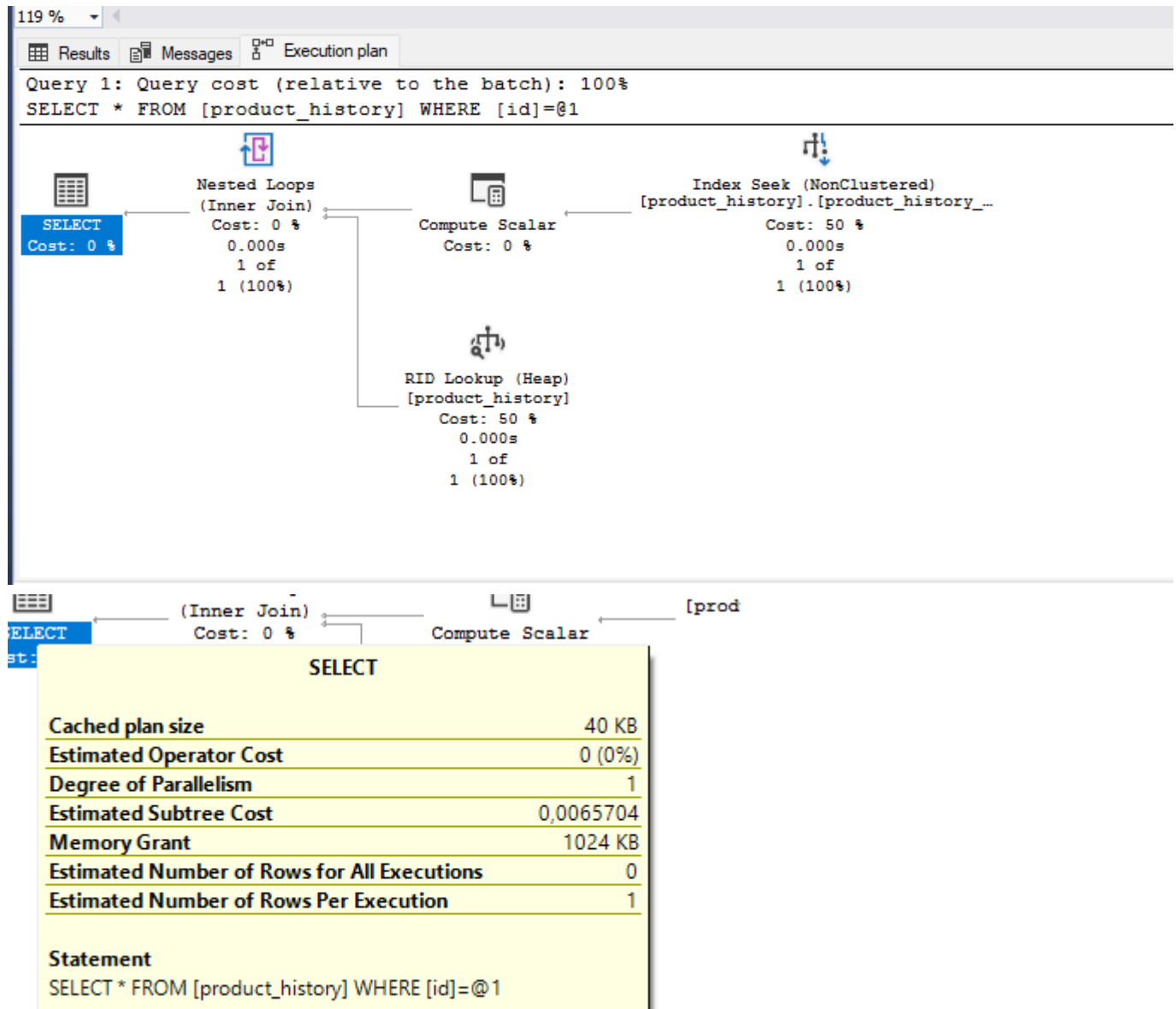
(1 row affected)

SQL Server Execution Times:
CPU time = 0 ms, elapsed time = 68 ms.

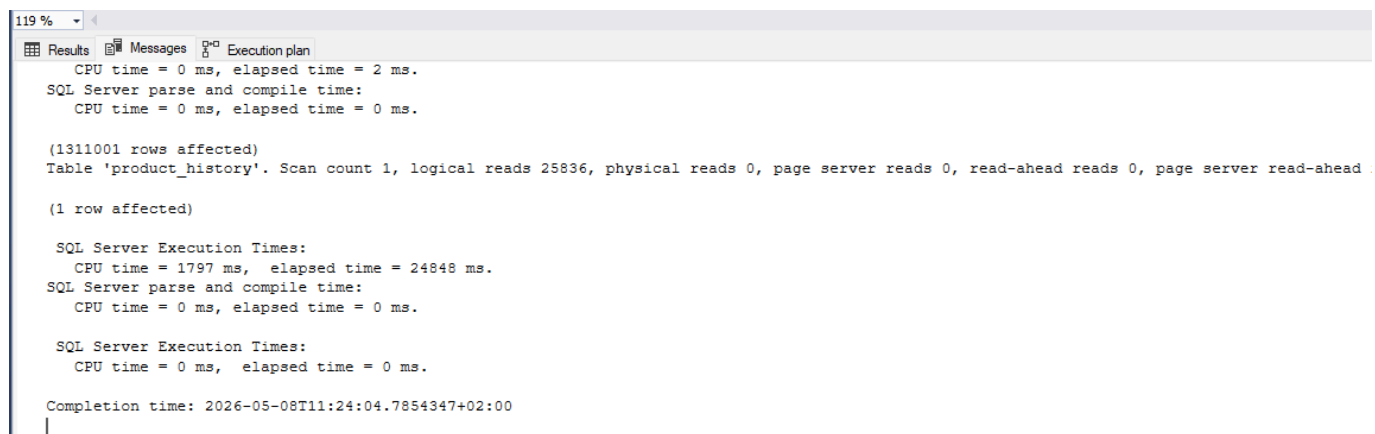
SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

SQL Server Execution Times:
CPU time = 0 ms, elapsed time = 0 ms.

Completion time: 2026-05-08T11:20:53.9761168+02:00



- drugie zapytanie



Query 1: Query cost (relative to the batch): 100%

```
SELECT * FROM [product_history] WHERE [id]>=@1 AND [id]<=@2
```

Missing Index (Impact 70.0375): CREATE NONCLUSTERED INDEX [<Name of Missing Index, sysname,>] ON [dbo].[product_history] ([id]) INCLUDE ([...])

Table Scan
[product_history]
Cost: 100 %
1.678s
1311001 of
1311000 (100%)

CREATE NONCLUSTERED INDEX [<Name of Missing Index, sysname,>]
ON [dbo].[product_history] ([id])
INCLUDE ([productid],[productname],[supplierid],[categoryid],[quantityperunit],[unitprice],[quantity],[value],[date])

SELECT	
Cached plan size	24 KB
Estimated Operator Cost	0 (0%)
Degree of Parallelism	1
Estimated Subtree Cost	21,6813
Estimated Number of Rows for All Executions	0
Estimated Number of Rows Per Execution	1311000

Statement
SELECT * FROM [product_history] WHERE [id]>=@1 AND [id]<=@2

Zapytanie	Plan	Logical reads	Estimated Cost	CPU / Elapsed
a1) <code>count(*) where id = 1000000</code>	Index Seek (NonClustered)	3	0,003	0 / 34 ms
a2) <code>count(*) where id between 999000 and 10000000</code>	Index Seek (NonClustered)	2934	4,40	171 / 224 ms
b1) <code>select * where id = 1000000</code>	Index Seek + RID Lookup (Heap) + Nested Loops	4	0,007	0 / 68 ms
b2) <code>select * where id between 999000 and 10000000</code>	Table Scan	25 836	-	1797 / 24 848 ms

Indeks nieklastrowy dla zapytań `where id = 1000000` z `count()` zredukował *logical reads* do 3, a z `select *` do 4. Dla zapytań `where id between 999000 and 10000000` z `count()` *logical reads* spadły do 2 934, ale z `select * SQL Server` zignorował indeks i wykonał Table Scan (25 836 *logical reads*). Czas wykonania zapytań `where id = 1000000` spadł niemal do zera (34 ms / 68 ms). Dla zapytań `where id between 999000 and 10000000` z `count(*)` *elapsed time* wynosi 224 ms, a z `select *` 24,8 s.

Indeks klastrowy jest lepszy do `select`, *nieklastrowy* do `count()`.

po zakończeniu pozostaw indeks klastrowy

d)

indeks dla kolumny **date**

```
create index product_history_date_idx
on product_history(date)

drop index product_history_date_idx on product_history
```

porównaj polecenia

```
select id, productid, productname, date
from product_history
where date >= '2001-01-01' and date <= '2001-01-31'

select id, productid, productname, date
from product_history
where year(date) = 2001 and month(date) = 1

select id, productid, productname, date
from product_history
where date >= '2001-01-01' and date <= '2001-12-31'

select id, productid, productname, date
from product_history
where year(date) = 2001
```

podczas analiz sprawdzaj jak zachowują się zapytania, zwróć uwagę na

- plan
- indeksy i sposób ich użycia
- koszt
- czas (ewentualnie, jeśli coś da się zaobserwować)
- liczbę odczytywanych stron !!!!

spróbuj skomentować wyniki tych analiz, dlaczego tak się dzieje

bez indeksu dla kolumny **date**

- pierwsze zapytanie

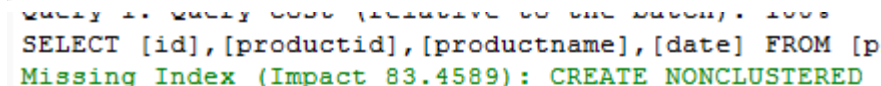
```
19 %
Results Messages Execution plan

(2387 rows affected)
Table 'product_history'. Scan count 5, logical reads 26051, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0

(1 row affected)

SQL Server Execution Times:
    CPU time = 266 ms,  elapsed time = 206 ms.
SQL Server parse and compile time:
    CPU time = 0 ms, elapsed time = 0 ms.

SQL Server Execution Times:
    CPU time = 0 ms, elapsed time = 0 ms.
```



- drugie zapytanie

119 %

Results Messages Execution plan

	id	productid	productname	date
1	1715561	1	Chai	2001-01-01
2	1715562	2	Chang	2001-01-01
3	1715563	3	Aniseed Syrup	2001-01-01
4	1715564	4	Chef Anton's Cajun Seasoning	2001-01-01
5	1715565	5	Chef Anton's Gumbo Mix	2001-01-01
6	1715566	6	Grandma's Boysenberry Spread	2001-01-01
7	1715567	7	Uncle Bob's Organic Dried Pears	2001-01-01
8	1715568	8	Northwoods Cranberry Sauce	2001-01-01
9	1715569	9	Mishi Kobe Niku	2001-01-01
10	1715570	10	Ikura	2001-01-01
11	1715571	11	Queso Cabrales	2001-01-01
12	1715572	12	Queso Manchego La Pastora	2001-01-01

119 %

Results Messages Execution plan

CPU time = 0 ms, elapsed time = 2 ms.

(2387 rows affected)

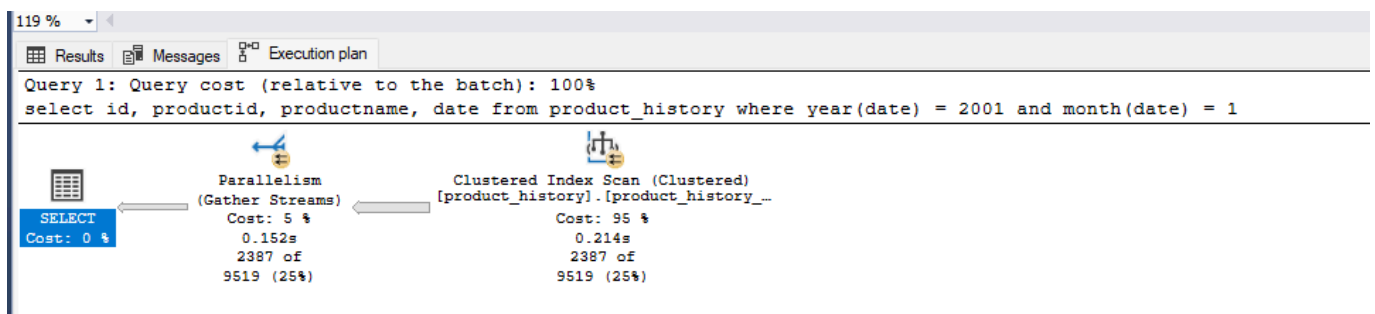
Table 'product_history'. Scan count 5, logical reads 26051, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0.

(1 row affected)

SQL Server Execution Times:
CPU time = 532 ms, elapsed time = 274 ms.

SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

SQL Server Execution Times:
CPU time = 0 ms, elapsed time = 0 ms.



SELECT	
Cached plan size	24 KB
Estimated Operator Cost	0 (0%)
Degree of Parallelism	4
Estimated Subtree Cost	21,5971
Memory Grant	72 KB
Estimated Number of Rows for All Executions	0
Estimated Number of Rows Per Execution	9519,21
Statement	
select id, productid, productname, date	
from product_history	
where year(date) = 2001 and month(date) = 1	

- trzecie zapytanie

119 %

Results Messages Execution plan

	id	productid	productname	date
1	1715561	1	Chai	2001-01-01
2	1715562	2	Chang	2001-01-01
3	1715563	3	Aniseed Syrup	2001-01-01
4	1715564	4	Chef Anton's Cajun Seasoning	2001-01-01
5	1715565	5	Chef Anton's Gumbo Mix	2001-01-01
6	1715566	6	Grandma's Boysenberry Spread	2001-01-01
7	1715567	7	Uncle Bob's Organic Dried Pears	2001-01-01
8	1715568	8	Northwoods Cranberry Sauce	2001-01-01
9	1715569	9	Mishi Kobe Niku	2001-01-01
10	1715570	10	Ikura	2001-01-01
11	1715571	11	Queso Cabrales	2001-01-01
12	1715572	12	Queso Manchego La Pastora	2001-01-01

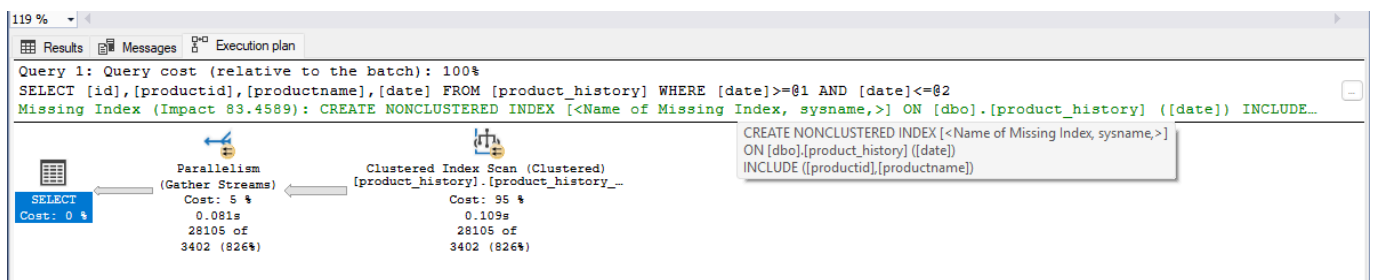
119 %

Results Messages Execution plan

CPU time = 0 ms, elapsed time = 0 ms.
 SQL Server parse and compile time:
 CPU time = 0 ms, elapsed time = 0 ms.

(28105 rows affected)
 Table 'product_history'. Scan count 5, logical reads 26051, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0.
 (1 row affected)

SQL Server Execution Times:
 CPU time = 298 ms, elapsed time = 445 ms.
 SQL Server parse and compile time:
 CPU time = 0 ms, elapsed time = 0 ms.



Missing Index (Impact 83.4589): CREATE NONCLUSTERED INDEX [<Name of Missing Index, sysname>] ON [dbo].[product_history] ([date]) INCLUDE ([productid],[productname])

SELECT	
Cached plan size	24 KB
Estimated Operator Cost	0 (0%)
Degree of Parallelism	4
Estimated Subtree Cost	21,4649
Memory Grant	72 KB
Estimated Number of Rows for All Executions	0
Estimated Number of Rows Per Execution	3402,06

Statement
 SELECT [id],[productid],[productname],[date] FROM [product_history] WHERE [date]>=@1 AND [date]<=@2

- czwarte zapytanie

119 %

Results Messages Execution plan

	id	productid	productname	date
1	1715561	1	Chai	2001-01-01
2	1715562	2	Chang	2001-01-01
3	1715563	3	Aniseed Syrup	2001-01-01
4	1715564	4	Chef Anton's Cajun Seasoning	2001-01-01
5	1715565	5	Chef Anton's Gumbo Mix	2001-01-01
6	1715566	6	Grandma's Boysenberry Spread	2001-01-01
7	1715567	7	Uncle Bob's Organic Dried Pears	2001-01-01
8	1715568	8	Northwoods Cranberry Sauce	2001-01-01
9	1715569	9	Mishi Kobe Niku	2001-01-01
10	1715570	10	Ikura	2001-01-01
11	1715571	11	Queso Cabrales	2001-01-01
12	1715572	12	Queso Manchego La Pastora	2001-01-01
13	1715573	13	Konbu	2001-01-01
14	1715574	14	Tofu	2001-01-01
15	1715575	15	Genen Shouyu	2001-01-01
16	1715576	16	Paulina	2001-01-01

119 %

Results Messages Execution plan

CPU time = 1 ms, elapsed time = 1 ms.

(28105 rows affected)

Table 'product_history'. Scan count 5, logical reads 26051, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead r

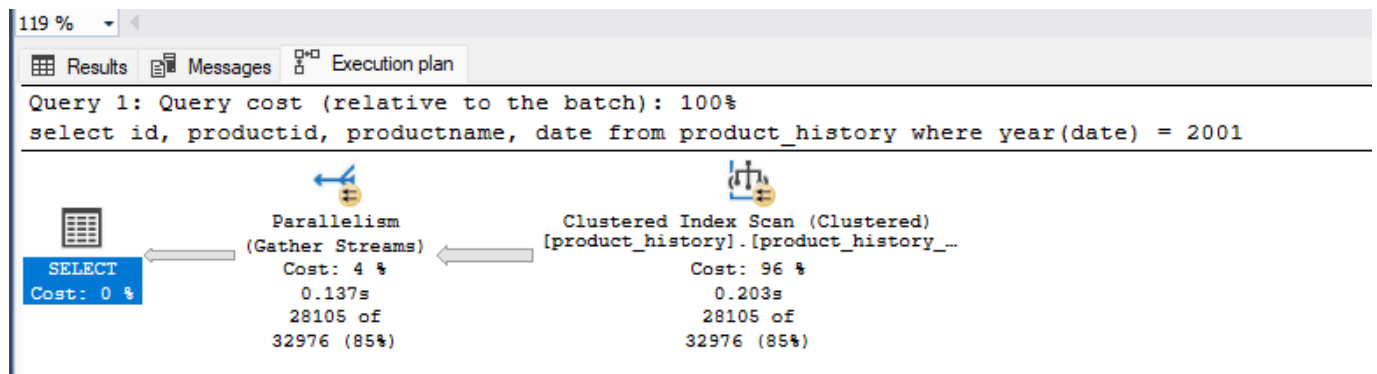
(1 row affected)

SQL Server Execution Times:
CPU time = 548 ms, elapsed time = 520 ms.

SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

SQL Server Execution Times:
CPU time = 0 ms, elapsed time = 0 ms.

Completion time: 2026-05-08T11:41:25.5261152+02:00



SELECT

Cached plan size	24 KB
Estimated Operator Cost	0 (0%)
Degree of Parallelism	4
Estimated Subtree Cost	21,1992
Memory Grant	72 KB
Estimated Number of Rows for All Executions	0
Estimated Number of Rows Per Execution	32975,5

Statement

```

select id, productid, productname, date
from product_history
where year(date) = 2001
  
```

Zapytanie	Plan	Logical reads	Estimated Cost	CPU / Elapsed
1. <code>where date >= '2001-01-01'</code> <code>and date <= '2001-01-31'</code>	Clustered Index Scan + Parallelism	26 051	21,46	266 / 206 ms
2. <code>where year(date) = 2001 and</code> <code>month(date) = 1</code>	Clustered Index Scan + Parallelism	26 051	21,60	532 / 274 ms
3. <code>where date >= '2001-01-01'</code> <code>and date <= '2001-12-31'</code>	Clustered Index Scan + Parallelism	26 051	21,46	298 / 445 ms
4. <code>where year(date) = 2001</code>	Clustered Index Scan + Parallelism	26 051	21,59	548 / 520 ms

z indeksem dla kolumny `date`

- pierwsze zapytanie

119 %

Results

Messages

Execution plan

	id	productid	productname	date
1	1715561	1	Chai	2001-01-01
2	1715562	2	Chang	2001-01-01
3	1715563	3	Aniseed Syrup	2001-01-01
4	1715564	4	Chef Anton's Cajun Seasoning	2001-01-01
5	1715565	5	Chef Anton's Gumbo Mix	2001-01-01
6	1715566	6	Grandma's Boysenberry Spread	2001-01-01
7	1715567	7	Uncle Bob's Organic Dried Pears	2001-01-01
8	1715568	8	Northwoods Cranberry Sauce	2001-01-01
9	1715569	9	Mishi Kobe Niku	2001-01-01
10	1715570	10	Ikura	2001-01-01
11	1715571	11	Queso Cabrales	2001-01-01
12	1715572	12	Queso Manchego La Pastora	2001-01-01

119 %

Results

Messages

Execution plan

CPU time = 0 ms, elapsed time = 0 ms.

(2387 rows affected)

Table 'product_history'. Scan count 1, logical reads 7327, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob...

Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob...

(1 row affected)

SQL Server Execution Times:

CPU time = 0 ms, elapsed time = 109 ms.

SQL Server parse and compile time:

CPU time = 0 ms, elapsed time = 0 ms.

SQL Server Execution Times:

119 %

Results Messages Execution plan

Query 1: Query cost (relative to the batch): 100%

```
SELECT [id],[productid],[productname],[date] FROM [product_history] WHERE [date]>=@1 AND [date]<=@2
```

Missing Index (Impact 53.275): CREATE NONCLUSTERED INDEX [<Name of Missing Index, sysname,>] ON [dbo].[product_history] ([<date>]) INCLUDE ([<productid>],[<productname>])

CREATE NONCLUSTERED INDEX [<Name of Missing Index, sysname,>] ON [dbo].[product_history] ([<date>]) INCLUDE ([<productid>],[<productname>])

Nested Loops (Inner Join)
Cost: 0 %
0.012s
2387 of
2387 (100%)

Index Seek (NonClustered)
[product_history].[product_history_...]
Cost: 0 %
0.003s
2387 of
2387 (100%)

Key Lookup (Clustered)
[product_history].[product_history_...]
Cost: 100 %
0.004s
2387 of

(Inner Join) [product_history].

SELECT

Cached plan size	40 KB
Estimated Operator Cost	0 (0%)
Degree of Parallelism	1
Estimated Subtree Cost	7,51793
Memory Grant	1024 KB
Estimated Number of Rows for All Executions	0
Estimated Number of Rows Per Execution	2387

Statement

```
SELECT [id],[productid],[productname],[date] FROM [product_history] WHERE [date]>=@1 AND [date]<=@2
```

- drugie zapytanie

119 %

Results Messages Execution plan

	id	productid	productname	date
1	1715561	1	Chai	2001-01-01
2	1715562	2	Chang	2001-01-01
3	1715563	3	Aniseed Syrup	2001-01-01
4	1715564	4	Chef Anton's Cajun Seasoning	2001-01-01
5	1715565	5	Chef Anton's Gumbo Mix	2001-01-01
6	1715566	6	Grandma's Boysenberry Spread	2001-01-01
7	1715567	7	Uncle Bob's Organic Dried Pears	2001-01-01
8	1715568	8	Northwoods Cranberry Sauce	2001-01-01
9	1715569	9	Mishi Kobe Niku	2001-01-01
10	1715570	10	Ikura	2001-01-01
11	1715571	11	Queso Cabrales	2001-01-01
12	1715572	12	Queso Manchego La Pastora	2001-01-01

119 %

Results Messages Execution plan

CPU time = 0 ms, elapsed time = 0 ms.

SQL Server parse and compile time:

CPU time = 0 ms, elapsed time = 3 ms.

(2387 rows affected)

Table 'product_history'. Scan count 5, logical reads 26051, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead r

(1 row affected)

SQL Server Execution Times:

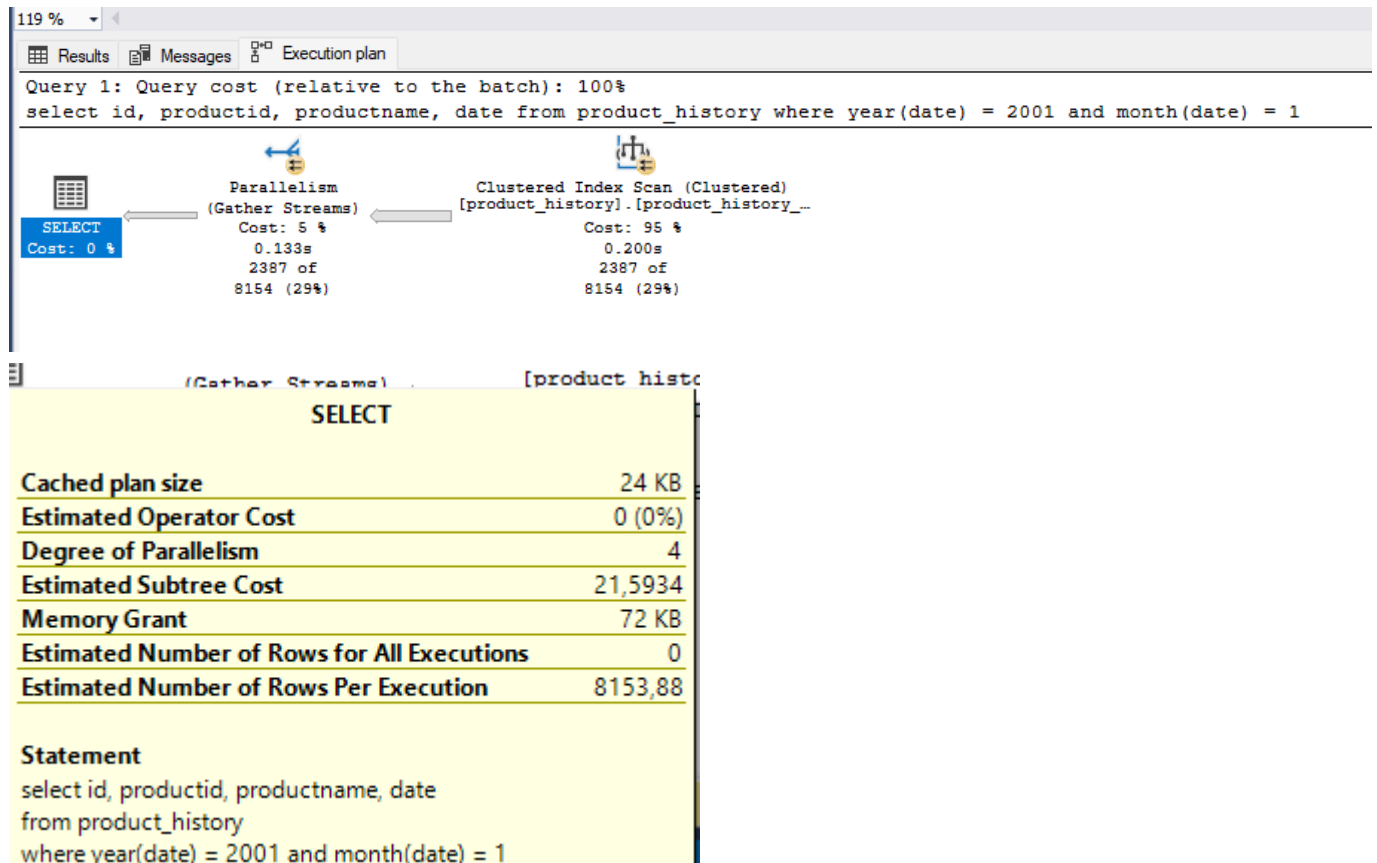
CPU time = 625 ms, elapsed time = 260 ms.

SQL Server parse and compile time:

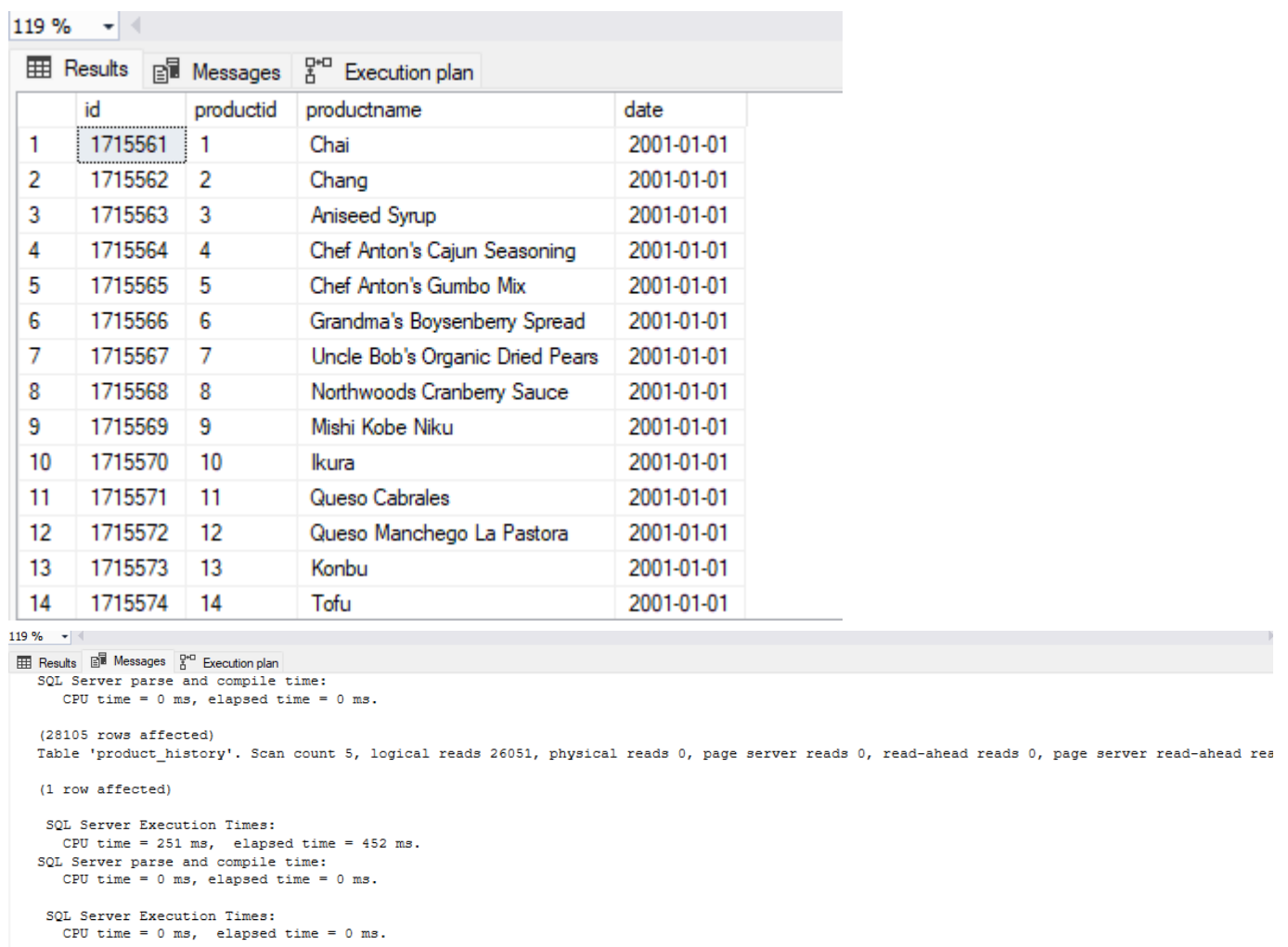
CPU time = 0 ms, elapsed time = 0 ms.

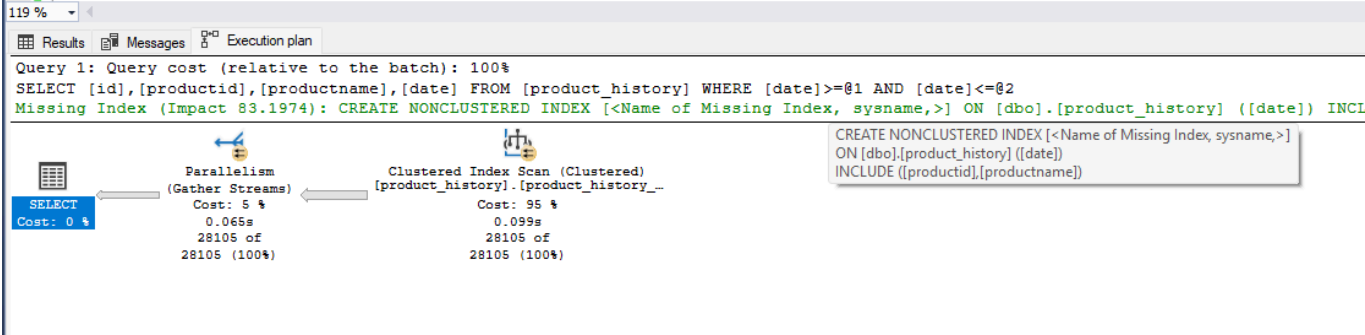
SQL Server Execution Times:

CPU time = 0 ms, elapsed time = 0 ms.



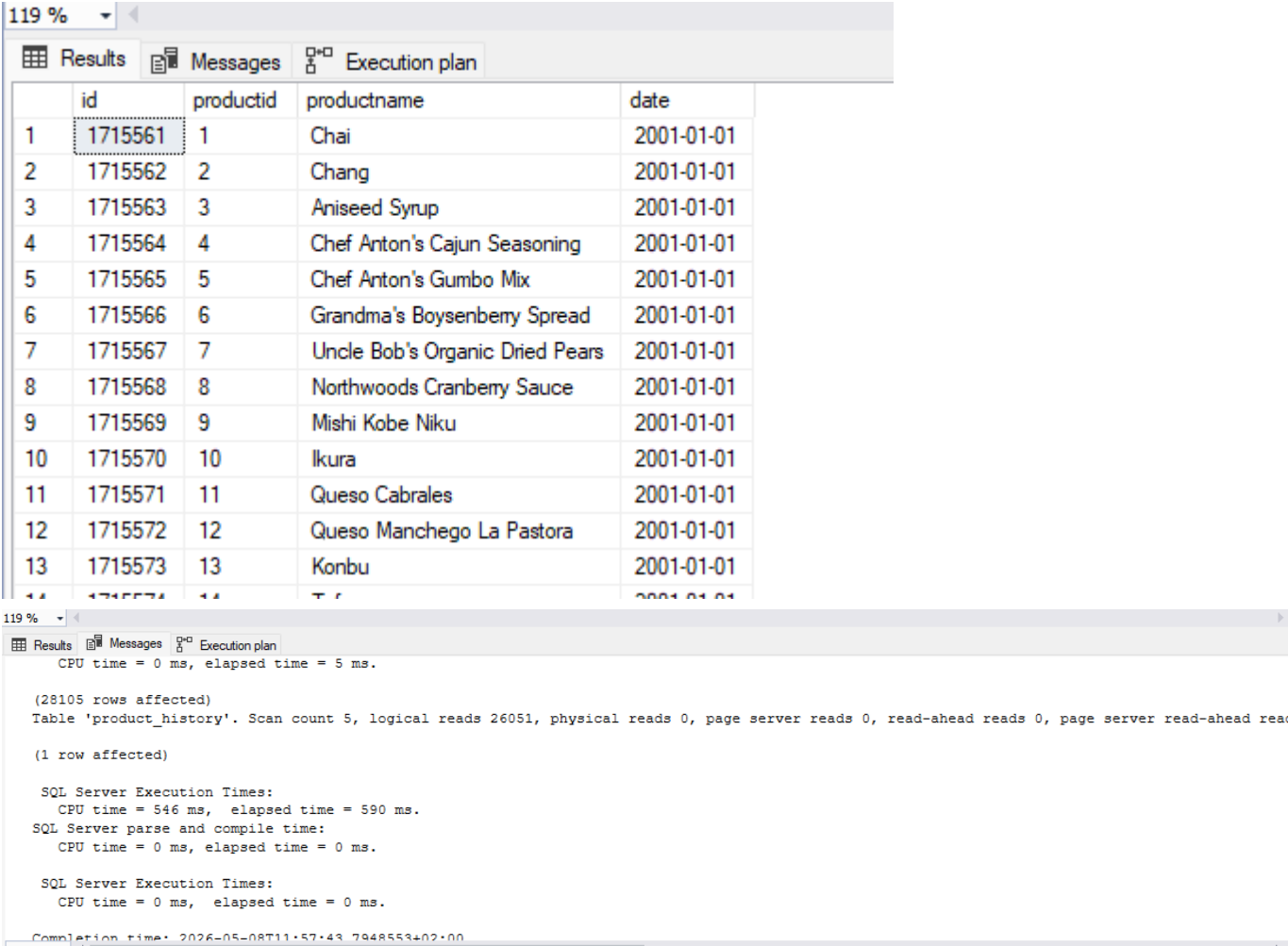
- trzecie zapytanie

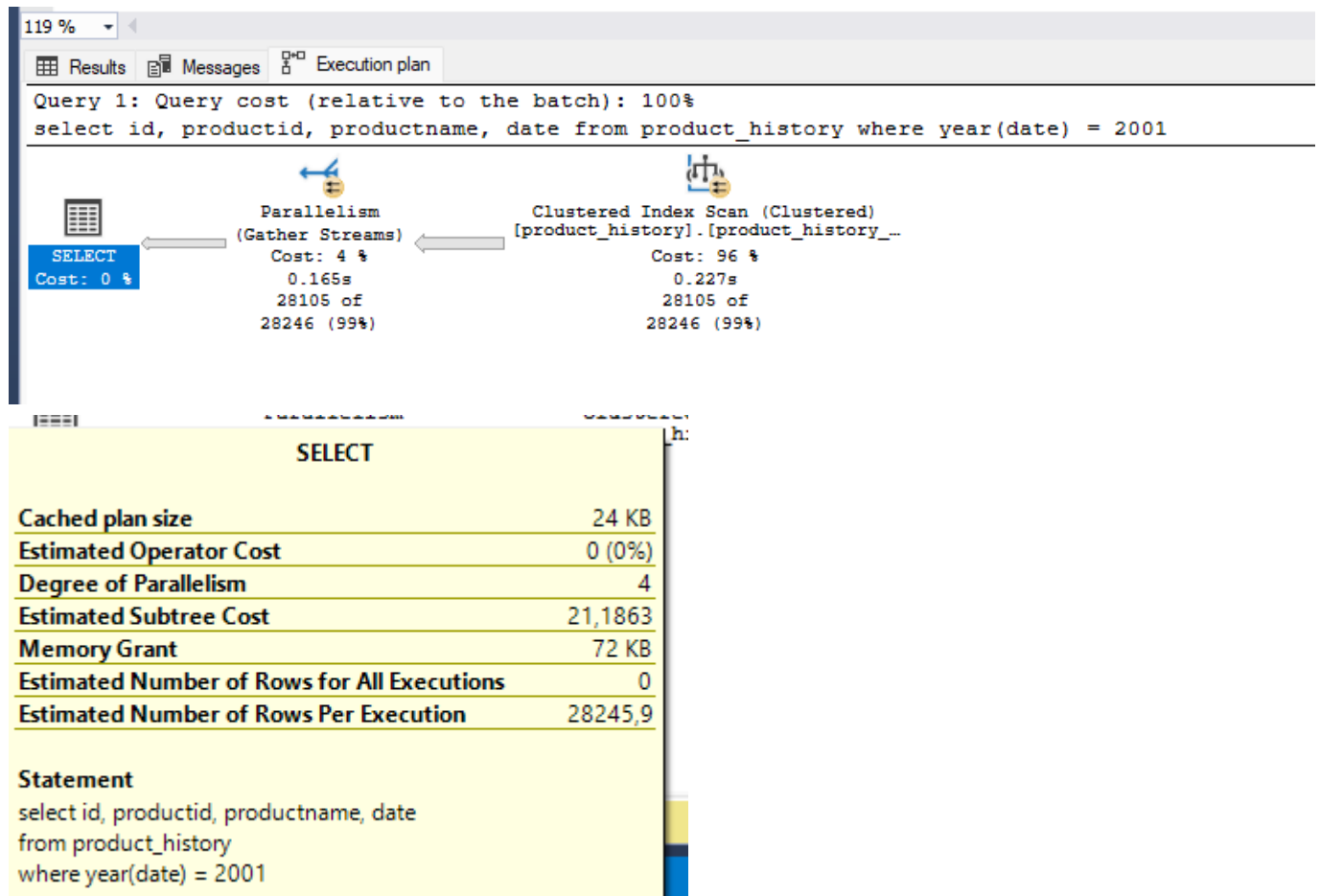




SELECT	
Cached plan size	24 KB
Estimated Operator Cost	0 (0%)
Degree of Parallelism	4
Estimated Subtree Cost	21,5324
Memory Grant	72 KB
Estimated Number of Rows for All Executions	0
Estimated Number of Rows Per Execution	28105
Statement	
SELECT [id],[productid],[productname],[date] FROM [product_history] WHERE [date]>=@1 AND [date]<=@2	

- czwarte zapytanie





Zapytanie	Plan	Logical reads	Estimated Cost	CPU / Elapsed
1. where date >= '2001-01-01' and date <= '2001-01-31'	Index Seek + Key Lookup (Nested Loops)	7 327	7,52	0 / 109 ms
2. where year(date) = 2001 and month(date) = 1	Clustered Index Scan + Parallelism	26 051	21,59	625 / 260 ms
3. where date >= '2001-01-01' and date <= '2001-12-31'	Clustered Index Scan + Parallelism	26 051	21,53	251 / 452 ms
4. where year(date) = 2001	Clustered Index Scan + Parallelism	26 051	21,19	546 / 590 ms

Indeks na date pomógł tylko zapytaniu nr 1, SQL Server wykonał Index Seek + Key Lookup i zredukował logical reads z 26 051 do 7 327. Pozostałe trzy zapytania zignorowały indeks, czas wykonania jest porównywalny do wersji bez indeksu. Pokazuje się Missing Index, który sugeruje indeks z INCLUDE (productid, productname) w zapytaniu 3 z indeksem date.

e)

powtórz eksperymenty z pkt d) , ale tym razem użyj indeksu zawierającego dodatkowe kolumny

```
create index product_history_date_incl_idx
on product_history(date) include(productid, productname)
```

```
drop index product_history_date_incl_idx on product_history
```

co się zmieniło?

- pierwsze zapytanie

	id	productid	productname	date
1	1715561	1	Chai	2001-01-01
2	1715562	2	Chang	2001-01-01
3	1715563	3	Aniseed Syrup	2001-01-01
4	1715564	4	Chef Anton's Cajun Seasoning	2001-01-01
5	1715565	5	Chef Anton's Gumbo Mix	2001-01-01
6	1715566	6	Grandma's Boysenberry Spread	2001-01-01
7	1715567	7	Uncle Bob's Organic Dried Pears	2001-01-01
8	1715568	8	Northwoods Cranberry Sauce	2001-01-01
9	1715569	9	Mishi Kobe Niku	2001-01-01

119 %

Results Messages Execution plan

SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 2 ms.
SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

(2387 rows affected)
Table 'product_history'. Scan count 1, logical reads 16, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead

(1 row affected)

SQL Server Execution Times:
CPU time = 32 ms, elapsed time = 103 ms.
SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

119 %

Results Messages Execution plan

Query 1: Query cost (relative to the batch): 100%
SELECT [id],[productid],[productname],[date] FROM [product_history] WHERE [date]>=@1 AND [date]<=@2

Index Seek (NonClustered)
[product_history].[product_history_--]
Cost: 100 %
0.001s
2387 of
2387 (100%)

SELECT	
Cached plan size	24 KB
Estimated Operator Cost	0 (0%)
Degree of Parallelism	1
Estimated Subtree Cost	0,0140558
Estimated Number of Rows for All Executions	0
Estimated Number of Rows Per Execution	2387
Statement	
SELECT [id],[productid],[productname],[date] FROM [product_history] WHERE [date]>=@1 AND [date]<=@2	

- drugie zapytanie

119 %

Results Messages Execution plan

	id	productid	productname	date
1	1715561	1	Chai	2001-01-01
2	1715562	2	Chang	2001-01-01
3	1715563	3	Aniseed Syrup	2001-01-01
4	1715564	4	Chef Anton's Cajun Seasoning	2001-01-01
5	1715565	5	Chef Anton's Gumbo Mix	2001-01-01
6	1715566	6	Grandma's Boysenberry Spread	2001-01-01
7	1715567	7	Uncle Bob's Organic Dried Pears	2001-01-01
8	1715568	8	Northwoods Cranberry Sauce	2001-01-01
9	1715569	9	Mishi Kobe Niku	2001-01-01
10	1715570	10	Ikura	2001-01-01
11	1715571	11	Queso Cabrales	2001-01-01
12	1715572	12	Queso Manchego La Pastora	2001-01-01

119 %

Results Messages Execution plan

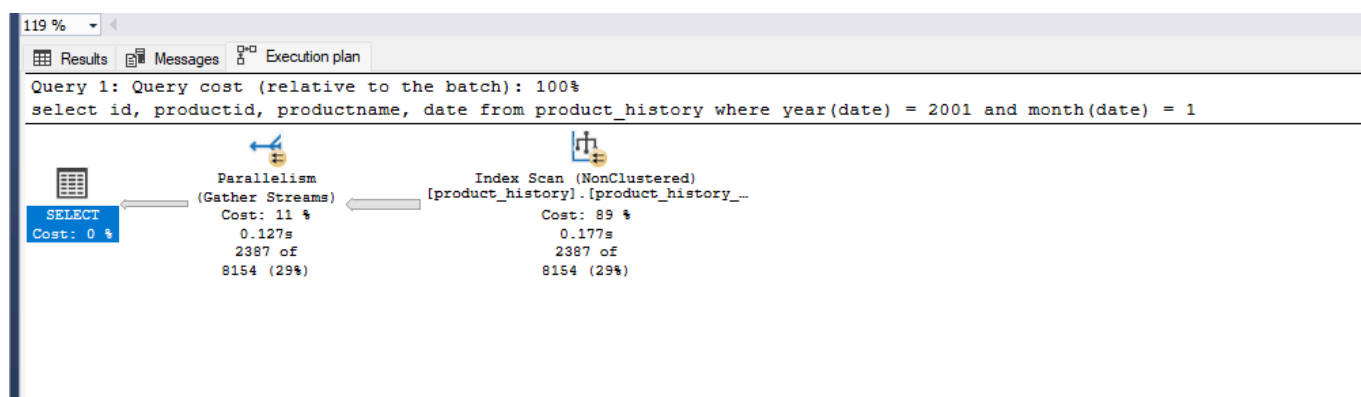
(2387 rows affected)
Table 'product_history'. Scan count 5, logical reads 11391, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0.

(1 row affected)

SQL Server Execution Times:
CPU time = 531 ms, elapsed time = 245 ms.

SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

SQL Server Execution Times:
CPU time = 0 ms, elapsed time = 0 ms.



SELECT

Cached plan size	24 KB
Estimated Operator Cost	0 (0%)
Degree of Parallelism	4
Estimated Subtree Cost	10,7897
Memory Grant	72 KB
Estimated Number of Rows for All Executions	0
Estimated Number of Rows Per Execution	8153,88

Statement

```

select id, productid, productname, date
from product_history
where year(date) = 2001 and month(date) = 1
  
```

- trzecie zapytanie

119 %

Results Messages Execution plan

	id	productid	productname	date
1	1715561	1	Chai	2001-01-01
2	1715562	2	Chang	2001-01-01
3	1715563	3	Aniseed Syrup	2001-01-01
4	1715564	4	Chef Anton's Cajun Seasoning	2001-01-01
5	1715565	5	Chef Anton's Gumbo Mix	2001-01-01
6	1715566	6	Grandma's Boysenberry Spread	2001-01-01
7	1715567	7	Uncle Bob's Organic Dried Pears	2001-01-01
8	1715568	8	Northwoods Cranberry Sauce	2001-01-01
9	1715569	9	Mishi Kobe Niku	2001-01-01
10	1715570	10	Ikura	2001-01-01
11	1715571	11	Queso Cabrales	2001-01-01
12	1715572	12	Queso Manchego La Pastora	2001-01-01
13	1715573	13	K... ..	2001-01-01

119 %

Results Messages Execution plan

CPU time = 0 ms, elapsed time = 0 ms.
SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

(28105 rows affected)
Table 'product_history'. Scan count 1, logical reads 142, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0

(1 row affected)

SQL Server Execution Times:
CPU time = 31 ms, elapsed time = 352 ms.
SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

119 %

Results Messages Execution plan

Query 1: Query cost (relative to the batch): 100%

SELECT [id],[productid],[productname],[date] FROM [product_history] WHERE [date]>=@1 AND [date]<=@2

Index Seek (NonClustered)
[product_history].[product_history_...]
Cost: 100 %
0.011s
28105 of
2387 (1177%)

SELECT
Cost: 0 %

SELECT

Cached plan size	24 KB
Estimated Operator Cost	0 (0%)
Degree of Parallelism	1
Estimated Subtree Cost	0,0140558
Estimated Number of Rows for All Executions	0
Estimated Number of Rows Per Execution	2387

Statement
SELECT [id],[productid],[productname],[date] FROM
[product_history] WHERE [date]>=@1 AND [date]<=@2

- czwarte zapytanie

119 %

Results Messages Execution plan

	id	productid	productname	date
1	1725726	2	Chang	2001-05-13
2	1725727	3	Aniseed Syrup	2001-05-13
3	1725728	4	Chef Anton's Cajun Seasoning	2001-05-13
4	1725729	5	Chef Anton's Gumbo Mix	2001-05-13
5	1725730	6	Grandma's Boysenberry Spread	2001-05-13
6	1725731	7	Uncle Bob's Organic Dried Pears	2001-05-13
7	1725732	8	Northwoods Cranberry Sauce	2001-05-13
8	1725733	9	Mishi Kobe Niku	2001-05-13
9	1725734	10	Ikura	2001-05-13
10	1725735	11	Queso Cabrales	2001-05-13
11	1725736	12	Queso Manchego La Pastora	2001-05-13
12	1725737	13	Konbu	2001-05-13

119 %

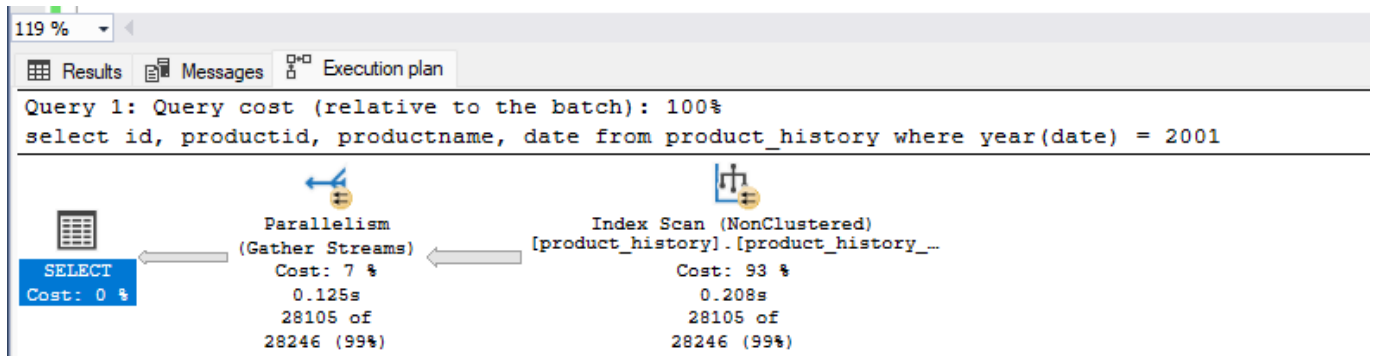
Results Messages Execution plan

CPU time = 0 ms, elapsed time = 0 ms.
 SQL Server parse and compile time:
 CPU time = 0 ms, elapsed time = 1 ms.

(28105 rows affected)
 Table 'product_history'. Scan count 5, logical reads 11391, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0.

(1 row affected)

SQL Server Execution Times:
 CPU time = 484 ms, elapsed time = 531 ms.
 SQL Server parse and compile time:
 CPU time = 0 ms, elapsed time = 0 ms.



SELECT	
Cached plan size	24 KB
Estimated Operator Cost	0 (0%)
Degree of Parallelism	4
Estimated Subtree Cost	10,3826
Memory Grant	72 KB
Estimated Number of Rows for All Executions	0
Estimated Number of Rows Per Execution	28245,9
Statement	
select id, productid, productname, date from product_history where year(date) = 2001	

Zapytanie	Plan	Logical reads	Estimated Cost	CPU / Elapsed
1. <code>where date >= '2001-01-01'</code> <code>and date <= '2001-01-31'</code>	Index Seek (NonClustered)	16	0,014	32 / 103 ms
2. <code>where year(date) = 2001</code> <code>and month(date) = 1</code>	Index Scan (NonClustered) + Parallelism	11 391	10,79	531 / 245 ms
3. <code>where date >= '2001-01-01'</code> <code>and date <= '2001-12-31'</code>	Index Seek (NonClustered)	142	0,014	31 / 352 ms
4. <code>where year(date) = 2001</code>	Index Scan (NonClustered) + Parallelism	11 391	10,38	484 / 531 ms

Indeks z INCLUDE (productid, productname) pomógł zapytaniom nr 1 i 3, jest Index Seek i logical reads zredukował się do 16 i 142. Zapytania nr 2 i 4 wykonują Index Scan, logical reads spadły do 11 391 z 26 051. Czas wykonania jest porównywalny do wersji z indeksem 'date'.

f)

indeks dla kolumny `categoryid`

```
create index product_history_cat_idx
on product_history(categoryid)

drop index product_history_cat_idx on product_history
```

przeanalizuj polecenia

```
select id, productid, productname, date
from product_history p
where categoryid = 8

select id, productid, productname, date, categoryname
from product_history p join categories c on p.categoryid = c.categoryid
where p.categoryid = 8
```

- pierwsze zapytanie

119 %

Results Messages

	id	productid	productname	date
1	10	10	lkura	1940-01-02
2	13	13	Konbu	1940-01-02
3	18	18	Camarvon Tigers	1940-01-02
4	30	30	Nord-Ost Matjeshering	1940-01-02
5	36	36	Inlagd Sill	1940-01-02
6	37	37	Gravad lax	1940-01-02
7	40	40	Boston Crab Meat	1940-01-02
8	41	41	Jack's New England Clam Chowder	1940-01-02
9	45	45	Rogede sild	1940-01-02
10	46	46	Spegesild	1940-01-02
11	58	58	Escargots de Bourgogne	1940-01-02
12	73	73	Röd Kaviar	1940-01-02

119 %

Results Messages Execution plan

CPU time = 0 ms, elapsed time = 7 ms.
 SQL Server parse and compile time:
 CPU time = 0 ms, elapsed time = 0 ms.

(360000 rows affected)
 Table 'product_history'. Scan count 5, logical reads 25987, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead r

(1 row affected)

SQL Server Execution Times:
 CPU time = 954 ms, elapsed time = 7771 ms.
 SQL Server parse and compile time:
 CPU time = 0 ms, elapsed time = 0 ms.

SQL Server Execution Times:

119 %

Results Messages Execution plan

Query 1: Query cost (relative to the batch): 100%

```
SELECT [id],[productid],[productname],[date] FROM [product_history] [p] WHERE [categoryid]=@1
```

Missing Index (Impact 88.8237): CREATE NONCLUSTERED INDEX [<Name of Missing Index, sysname,>] ON [dbo].[product_history] ([categoryid]) IN...

CREATE NONCLUSTERED INDEX [<Name of Missing Index, sysname,>]
 ON [dbo].[product_history] ([categoryid])
 INCLUDE ([productid],[productname],[date])

Parallelism (Gather Streams) ← Clustered Index Scan (Clustered)
 [product_history].[product_history_...]
 Cost: 7 % 0.147s 360000 of 360000 (100%)
 Cost: 93 % 0.218s 360000 of 360000 (100%)

SELECT

Cached plan size	24 KB
Estimated Operator Cost	0 (0%)
Degree of Parallelism	4
Estimated Subtree Cost	21,9767
Memory Grant	72 KB
Estimated Number of Rows for All Executions	0
Estimated Number of Rows Per Execution	360000

Statement

```
SELECT [id],[productid],[productname],[date] FROM [product_history] [p] WHERE [categoryid]=@1
```

- drugie zapytanie

```

119 %
Results Messages Execution plan
CPU time = 0 ms, elapsed time = 3 ms.

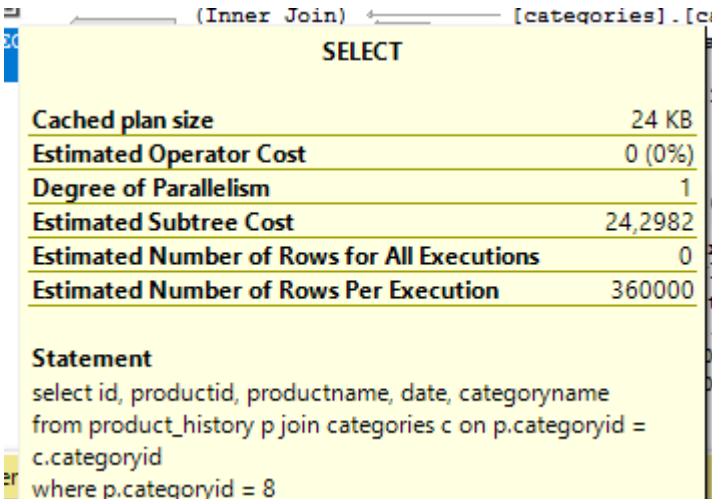
(360000 rows affected)
Table 'product_history'. Scan count 1, logical reads 25881, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, logical reads 25881, logical writes 0, physical writes 0, page server writes 0, read-ahead writes 0, page server read-ahead writes 0, logical writes 0.
Table 'categories'. Scan count 1, logical reads 2, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, logical reads 2, logical writes 0, physical writes 0, page server writes 0, read-ahead writes 0, page server read-ahead writes 0, logical writes 0.

(1 row affected)

SQL Server Execution Times:
    CPU time = 812 ms,  elapsed time = 9509 ms.
SQL Server parse and compile time:
    CPU time = 0 ms,  elapsed time = 0 ms.

SQL Server Execution Times:
    CPU time = 0 ms,  elapsed time = 0 ms.
119 %

```



Zapytanie	Plan	Logical reads	Estimated Cost	CPU / Elapsed
1. <code>where categoryid = 8</code>	Clustered Index Scan + Parallelism	25 987	21,98	954 / 771 ms
2. <code>JOIN where p.categoryid = 8</code>	Nested Loops + Clustered Index Scan + Clustered Index Seek (categories)	25 881 + 2 (categories)	24,30	812 / 9509 ms

W obu zapytaniach jest Clustered Index Scan. Logical reads są bardzo podobne (25 987 i 25 881), bo oba zapytania skanują tabelę product_history. Czas wykonania jest długi w obu (7 771 ms i 9 509 ms). Pokazuje się Missing Index, który sugeruje indeks z INCLUDE (id, productid, productname, date) na kolumnie categoryid.

dodatkowo

możesz sprawdzić strukturę indeksu

np.

```
exec sp_helpindex 'dbo.product_history';

select
    i.name as index_name,
    ips.index_depth,
    ips.index_level,
    ips.page_count
from sys.indexes i
cross apply sys.dm_db_index_physical_stats(
    db_id(),
    i.object_id,
    i.index_id,
    null,
    'detailed'
) ips
where i.object_id = object_id('dbo.product_history')
and i.name = 'product_history_date_idx';
```

119 %

Results Messages Execution plan

	index_name	index_description	index_keys
1	product_history_clust_idx	clustered located on PRIMARY	id
2	product_history_date_idx	nonclustered located on PRIMARY	date

	index_name	index_depth	index_level	page_count
1	product_history_date_idx	3	0	3716
2	product_history_date_idx	3	1	12
3	product_history_date_idx	3	2	1

119 %

Results Messages Execution plan

Query 7: Query cost (relative to the batch): 2%

```
select @groupname = name from sys.data_spaces where data_space_id = @groupid
```

Clustered Index Seek (Clustered)

SELECT [sysclsobjs].[clst] [o]

Cost: 0 % 0.000s 1 of 1 (100%)

Query 8: Query cost (relative to the batch): 11%

```
select @memory_optimized = is_memory_optimized from sys.tables where object_id = @objid
```

Execution plan diagram showing various operations: Nested Loops, Clustered Index Seek, Compute Scalar, Filter, and Nested Loops. Costs and statistics are provided for each step.

Query 9: Query cost (relative to the batch): 7%

```
insert into #spindtab values (@indname, @indid, @ignore_dup_key, @is_unique, @is_hypothetical, @is_primary_key, @is_unique_key, @is_colu...
```

Table Insert

INSERT [#spindtab]

Cost: 0 % 0.000s 1 of 1 (100%)

Indeks product_history_date_idx ma strukturę B-drzewa o głębokości 3 poziomów:

- poziom 0 (liście): 3716 stron
- poziom 1: 12 stron
- poziom 2 (root): 1 strona

jeśli chcesz zaobserwować odczyty logiczne/fizyczne możesz zwolnić pulę buforów przed wykonaniem polecenia

```
CHECKPOINT;
DBCC DROPCLEANBUFFERS;
```

i teraz porównaj liczby czytanych stron np. wykonując dwukrotnie polecenie

```
select * from product_history
```

```
119 %
Results Messages Execution plan
SQL Server parse and compile time:
  CPU time = 0 ms, elapsed time = 0 ms.

SQL Server Execution Times:
  CPU time = 0 ms, elapsed time = 0 ms.
SQL Server parse and compile time:
  CPU time = 0 ms, elapsed time = 4 ms.

(2310000 rows affected)
Table 'product_history'. Scan count 1, logical reads 25881, physical reads 1, page server reads 0, read-ahead reads 25892, page server read-ah:

(1 row affected)

SQL Server Execution Times:
  CPU time = 2593 ms, elapsed time = 36717 ms
```

```
119 %
Results Messages Execution plan
SQL Server Execution Times:
  CPU time = 0 ms, elapsed time = 0 ms.
SQL Server parse and compile time:
  CPU time = 0 ms, elapsed time = 0 ms.

(2310000 rows affected)
Table 'product_history'. Scan count 1, logical reads 25881, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead r:

(1 row affected)

SQL Server Execution Times:
  CPU time = 3891 ms, elapsed time = 45285 ms.
SQL Server parse and compile time:
  CPU time = 0 ms, elapsed time = 0 ms.
```

Wykonanie	logical reads	physical reads	read-ahead reads	CPU / Elapsed
1.	25 881	1	25 892	2 593 / 36 717 ms
2.	25 881	0	0	3 891 / 45 285 ms

Pierwsze wykonanie pokazuje physical reads = 1 i read-ahead reads = 25 892. Drugie wykonanie ma physical reads = 0 i read-ahead reads = 0. Logical reads są identyczne (25 881) niezależnie od tego, czy strony są w pamięci.

Zadanie 2

Celem zadania jest poznanie indeksów typu column store

Utwórz tabelę testową:

```
create table saleshistory(
  id int identity(1,1) not null primary key,
  salesorderid int not null,
  salesorderdetailid int not null,
  carriertrackingnumber nvarchar(25) null,
  orderqty smallint not null,
  productid int not null,
  specialofferid int not null,
  unitprice money not null,
  unitpricediscount money not null,
  linetotal numeric(38, 6) not null,
  rowguid uniqueidentifier not null,
  modifieddate datetime not null
)
```

Sprawdź jakie indeksy istnieją dla tej tabeli

```
exec sp_helpindex 'dbo.saleshistory'
```

120 %			
Results Messages			
	index_name	index_description	index_keys
1	PK_saleshis__3213E83F5586371F	clustered, unique, primary key located on PRIMARY	id

```
Select
    i.name as index_name,
    i.type_desc,
    i.is_unique,
    c.name as column_name,
    ic.key_ordinal,
    ic.is_included_column
from sys.indexes i
join sys.index_columns ic
    on i.object_id = ic.object_id
    and i.index_id = ic.index_id
join sys.columns c
    on ic.object_id = c.object_id
    and ic.column_id = c.column_id
where i.object_id = object_id('dbo.saleshistory')
order by i.name, ic.key_ordinal;
```

120 %						
Results Messages						
	index_name	type_desc	is_unique	column_name	key_ordinal	is_included_column
1	PK_saleshis__3213E83F5586371F	CLUSTERED	1	id	1	0

Wypełnij tablicę danymi:

```
-- W SSMS

insert into saleshistory
select sh.*
from adventureworks2017.sales.salesorderdetail sh
go 100
```

(UWAGA GO 100 oznacza 100 krotne wykonanie polecenia. Jeżeli podejrzewasz, że twój serwer może to zbyt przeciążyć, zacznij od GO 10, GO 20, GO 50

albo

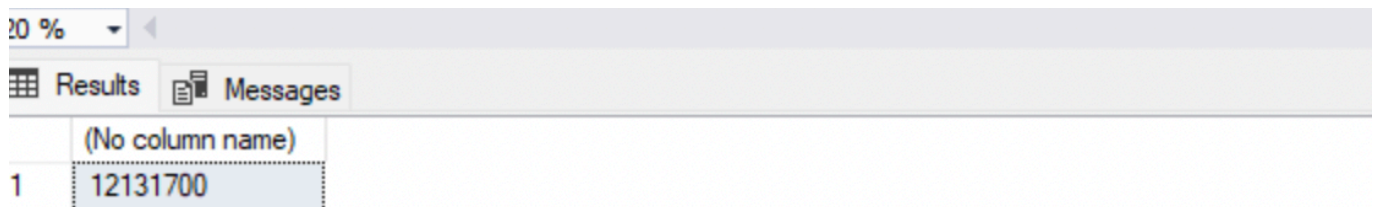
```
declare @i int = 1;

while @i <= 100
begin
    insert into saleshistory
    select *
    from adventureworks2017.sales.salesorderdetail;

    set @i += 1;
end;
```

sprawdź liczbę wierszy w tabeli

```
select count(*) from saleshistory
```



The screenshot shows the SQL Server Enterprise Manager interface. The 'Results' tab is active, displaying a single row of data. The first column is labeled '(No column name)' and the value is 12131700.

(No column name)
12131700

włącz statystyki IO i TIME

```
SET STATISTICS IO ON

SET STATISTICS TIME ON;
```

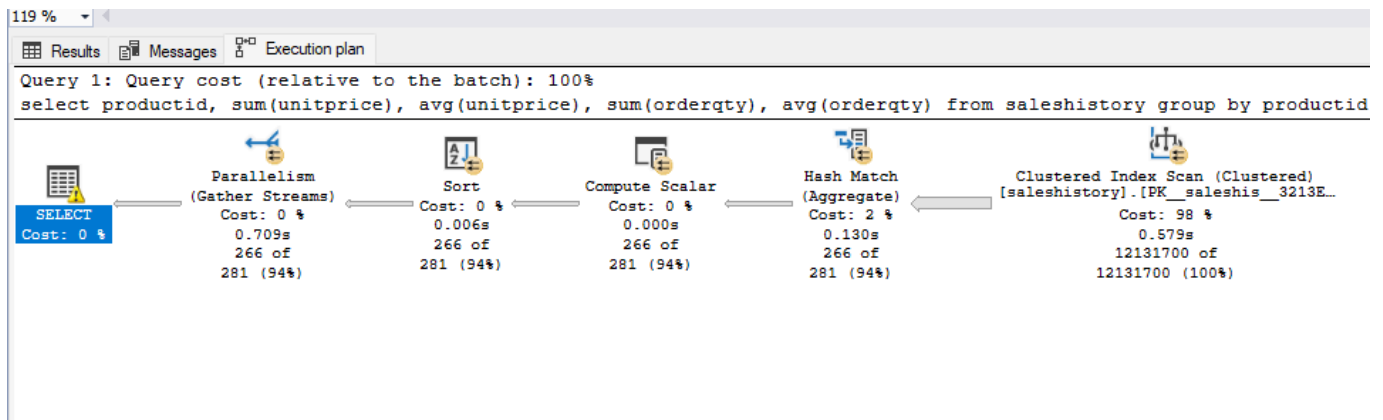
Sprawdź jak zachowa się zapytanie

- sprawdź plan
- koszt
- czas
- liczbę odczytywanych stron

```
select productid, sum(unitprice), avg(unitprice), sum(orderqty), avg(orderqty)
from saleshistory
group by productid
order by productid
```


119 %					
Results Messages Execution plan					
	productid	(No column name)	(No column name)	(No column name)	(No column name)
1	707	9522317.98	30,8865	626600	2
2	708	9156173.10	30,4495	653200	2
3	709	106333.50	5,656	110700	5
4	710	25080.00	5,70	9000	2
5	711	9382785.66	30,365	674300	2
6	712	2593395.71	7,6682	831100	2
7	713	2144571.00	49,99	42900	1
8	714	4479942.95	36,7811	363600	2
9	715	5706317.12	34,901	659200	4
10	716	3998954.56	37,165	298000	2
11	717	17746102.26	814,0413	48500	2
12	718	17810841.95	813,2804	48600	2
13	719	3615188.22	821,6336	10900	2
14	722	7383582.92	188,3567	94000	2
15	723	1003766.78	193,032	12900	2
16	725	7275245.88	194,5252	99100	2
17	726	5634271.96	195.6344	67100	2

119 %					
Results Messages Execution plan					
SQL Server parse and compile time:					
CPU time = 0 ms, elapsed time = 6 ms.					
(266 rows affected)					
Table 'saleshistory'. Scan count 5, logical reads 158276, physical reads 0, page server reads 0, read-ahead reads 307, page server read-ahead re					
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lok					
(1 row affected)					
SQL Server Execution Times:					
CPU time = 2437 ms, elapsed time = 827 ms.					
SQL Server parse and compile time:					
CPU time = 0 ms, elapsed time = 0 ms.					
SQL Server Execution Times:					
CPU time = 0 ms, elapsed time = 0 ms.					
Completion time: 2026-05-08T12:52:15.9268590+02:00					



SELECT	
Cached plan size	56 KB
Estimated Operator Cost	0 (0%)
Degree of Parallelism	4
Estimated Subtree Cost	124,711
Memory Grant	78 MB
Estimated Number of Rows for All Executions	0
Estimated Number of Rows Per Execution	281
Statement	
select productid, sum(unitprice), avg(unitprice), sum (orderqty), avg(orderqty) from saleshistory group by productid order by productid	
Warnings	
The query memory grant detected "ExcessiveGrant", which may impact the reliability. Grant size: Initial 79592 KB, Final 79592 KB, Used 3840 KB.	

Założ indeks typu column store:

```
create nonclustered columnstore index saleshistory_columnstore
on saleshistory(unitprice, orderqty, productid)
```

119 %

ResultsMessagesExecution plan

	productid	(No column name)	(No column name)	(No column name)	(No column name)
1	707	9522317,98	30,8865	626600	2
2	708	9156173,10	30,4495	653200	2
3	709	106333,50	5,656	110700	5
4	710	25080,00	5,70	9000	2
5	711	9382785,66	30,365	674300	2
6	712	2593395,71	7,6682	831100	2
7	713	2144571,00	49,99	42900	1
8	714	4479942,95	36,7811	363600	2
9	715	5706317,12	34,901	659200	4
10	716	3998954,56	37,165	298000	2
11	717	17746102,26	814,0413	48500	2
12	718	17810841,95	813,2804	48600	2
13	719	3615188,22	821,6336	10900	2
14	722	7383582,92	188,3567	94000	2

119 %

ResultsMessagesExecution plan

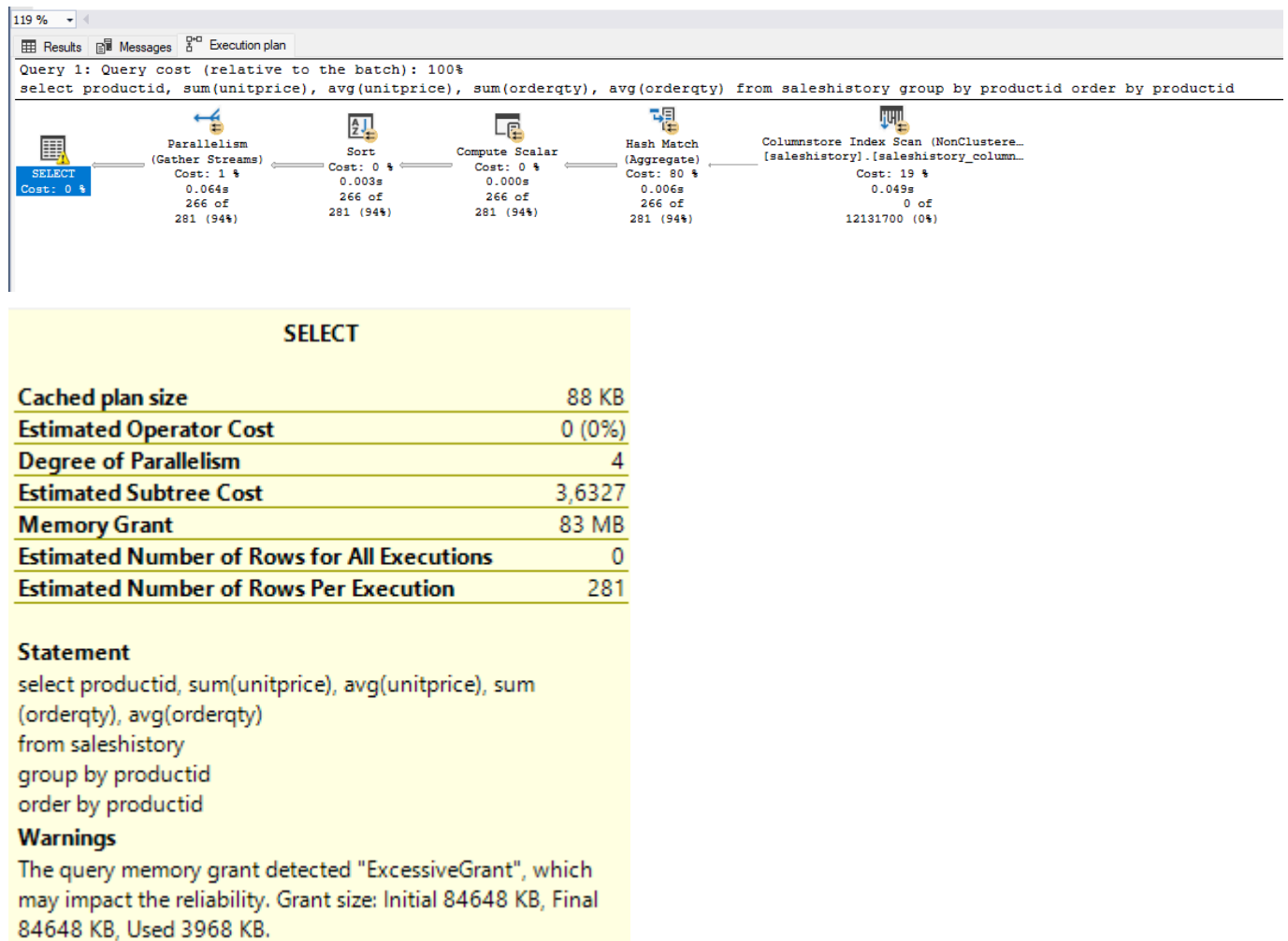
SQL Server Execution Times:
CPU time = 0 ms, elapsed time = 0 ms.
SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 8 ms.

(266 rows affected)
Table 'saleshistory'. Scan count 8, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob
Table 'saleshistory'. Segment reads 13, segment skipped 0.
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob

(1 row affected)

SQL Server Execution Times:
CPU time = 78 ms, elapsed time = 199 ms.
SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

119 %



Sprawdź różnicę pomiędzy przetwarzaniem w zależności od indeksów. Porównaj plany i opisz różnicę. Co to są indeksy column store? Jak działają? (poszukaj materiałów w internecie/literaturze)

Wykonanie	Plan	Logical reads	LOB logical reads	Estimated Cost	CPU / Elapsed
Bez columnstore	Clustered Index Scan + Hash Match (Aggregate) + Sort + Parallelism	158 276	0	124,71	2 437 / 827 ms
Z columnstore	Columnstore Index Scan + Hash Match + Sort + Parallelism	0	11 457	3,63	78 / 199 ms

Logical reads dla zapytania z columnstore spadły z 158 276 do 0, ale pojawiło się 11 457 LOB logical reads. Estimated Cost spadł z 124,71 do 3,63, a elapsed time z 827 ms do 199 ms. W zapytaniu bez columnstore jest Clustered Index Scan, a z columnstore występuje Columnstore Index Scan. W zapytaniu bez columnstore jest Clustered Index Scan, a z columnstore występuje Columnstore Index Scan.

<https://learn.microsoft.com/en-us/sql/relational-databases/indexes/columnstore-indexes-overview?view=sql-server-ver17>

Indeks columnstore to technologia służąca do przechowywania, pobierania i zarządzania danymi przy użyciu kolumnowego formatu danych, zwanego columnstore.

Jak działają:

1. Rowgroup - grupa wierszy kompresowanych razem do formatu columnstore, zwykle zawierająca maksymalną liczbę 1 048 576 wierszy. Indeks columnstore dzieli tabelę na rowgroups, a następnie kompresuje każdą z nich kolumnowo.
2. Column segment - kolumna danych w obrębie rowgroup. Każda rowgroup zawiera jeden segment kolumny dla każdej kolumny w tabeli, a każdy segment jest kompresowany razem i przechowywany na nośniku fizycznym. Każdy segment ma metadane pozwalające na szybką eliminację segmentów bez ich odczytywania.
3. Deltastore - aby zmniejszyć fragmentację segmentów kolumn i poprawić wydajność, indeks columnstore może tymczasowo przechowywać niektóre dane w klastrowanym indeksie zwanym deltastore oraz w liście B-tree identyfikatorów usuniętych wierszy.
4. Tuple-mover - proces w tle, który sprawdza zamknięte rowgroups - jeśli znajdzie taką grupę, kompresuje ją i zapisuje w columnstore jako rowgroup w stanie COMPRESSED.
5. Batch mode execution - metoda przetwarzania zapytań przetwarzająca wiele wierszy razem. Jest ściśle zintegrowana z formatem columnstore i zoptymalizowana wokół niego. Zapytania na indeksach columnstore używają trybu batch, co zwykle poprawia wydajność 2-4 razy.

Kiedy używać:

- Klastrowane indeksy columnstore do przechowywania tabel faktów i dużych tabel wymiarów w hurtowniach danych. Poprawia wydajność zapytań i kompresję danych do 10 razy.
- Nieklastrowane indeksy columnstore do analizy w czasie rzeczywistym na obciążeniu OLTP.

Powody, dla których indeksy columnstore są tak szybkie:

- Kolumny przechowują wartości z tej samej dziedziny i często mają podobne wartości, więc jest wysoki współczynnik kompresji. Wąskie gardła I/O w systemie są minimalizowane lub eliminowane, a zużycie pamięci jest znacząco zmniejszone.
- Wysokie współczynniki kompresji poprawiają wydajność zapytań poprzez mniejszy ślad pamięciowy. W rezultacie wydajność zapytań może się poprawić, ponieważ SQL Server może wykonywać więcej operacji na zapytaniach i danych w pamięci.
- Wykonywanie wsadowe (batch execution) poprawia wydajność zapytań zazwyczaj 2-4 krotnie, przetwarza wiele wierszy jednocześnie.
- Zapytania często wybierają tylko kilka kolumn z tabeli, co redukuje całkowite I/O z nośnika fizycznego.

UWAGA: ciekawsze efekty możesz zaobserwować dla jeszcze większych tabel (jeśli twój komp na to pozwala możesz zwiększyć wolumen generowanych danych)

Zadanie 3 – własne eksperymenty

Należy zaprojektować/zaimplementować tabelę w bazie danych, lub wybrać dowolny schemat/bazę/tabelę (poza używanymi na zajęciach), a następnie wypełnić ją danymi w taki sposób, przetestować/przeanalizować działanie indeksów różnego typu. Warto wygenerować sobie tabele o większym rozmiarze.

Możesz też powtórzyć np. eksperymenty wykonywane w zadaniu 1, ale tym razem dla innego serwera,

Wedle uznania i zainteresowań, ważne żeby poeksplorować tematykę i spróbować

Do analizy, proszę uwzględnić następujące rodzaje indeksów:

- Klastrowane (np. dla atrybutu nie będącego kluczem głównym)
- Nieklastrowane
- Indeksy wykorzystujące kilka atrybutów, indeksy include
- Filtered Index (Indeks warunkowy)
- Kolumnowe

Analiza

Proszę przygotować zestaw zapytań do danych, które:

- wykorzystują poszczególne indeksy
- które przy wymuszeniu indeksu działają gorzej, niż bez niego (lub pomimo założonego indeksu, tabela jest w pełni skanowana) Odpowiedź powinna zawierać:
- Schemat tabeli
- Opis danych (ich rozmiar, zawartość, statystyki)
- Opis indeksu
- Przygotowane zapytania, wraz z wynikami z planów (zrzuty ekranów)
- Inf o kosztach, czytanych stornach
- Komentarze do zapytań, ich wyników
- ew. sprawdzenie, co proponuje Database Engine Tuning Advisor (porównanie czy udało się Państwu znaleźć odpowiednie indeksy do zapytania)

Wyniki:

Schemat tabeli

Tabela **OrdersTest** przechowuje przykładowe dane zamówień sklepu internetowego. Zawiera informacje o klientach, produktach, datach zamówień, statusach oraz cenach produktów. Tabela została wykorzystana do analizy wydajności różnych typów indeksów w SQL Server.

```
create table OrdersTest
(
    OrderID INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Order PRIMARY KEY
NONCLUSTERED
    CustomerID int,
    ProductID int,
    OrderDate datetime,
    Status varchar(20),
    Quantity int,
    Price money,
    City varchar(50)
)

set nocount on

declare @i int = 1
```

```
while @i <= 500000
begin

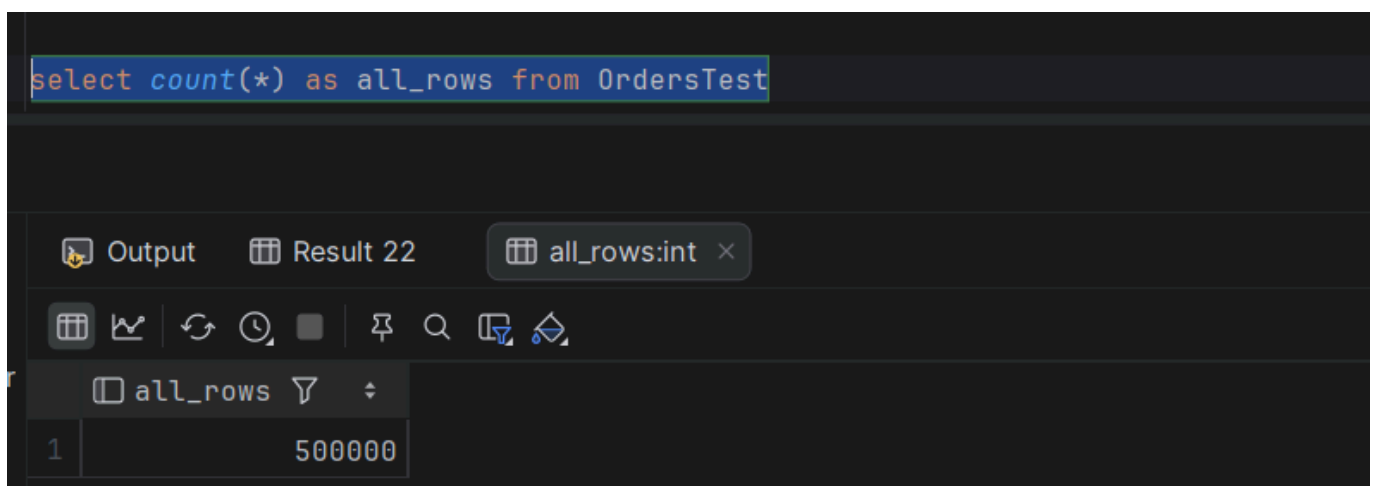
    insert into OrdersTest
    (
        CustomerID,
        ProductID,
        OrderDate,
        Status,
        Quantity,
        Price,
        City
    )
    values
    (
        abs(checksum(newid())) % 10000,
        abs(checksum(newid())) % 5000,
        dateadd(day, -abs(checksum(newid())) % 1000, getdate()),

        case
            when @i % 10 = 0 then 'Cancelled'
            when @i % 5 = 0 then 'Pending'
            else 'Completed'
        end,

        abs(checksum(newid())) % 20 + 1,
        abs(checksum(newid())) % 5000 + 100,

        case
            when @i % 3 = 0 then 'Warsaw'
            when @i % 3 = 1 then 'Krakow'
            else 'Gdansk'
        end
    )
    set @i += 1
end
```

Jak widać poprawnie wygenerowaliśmy 0,5 mln rekordów.



Opis danych

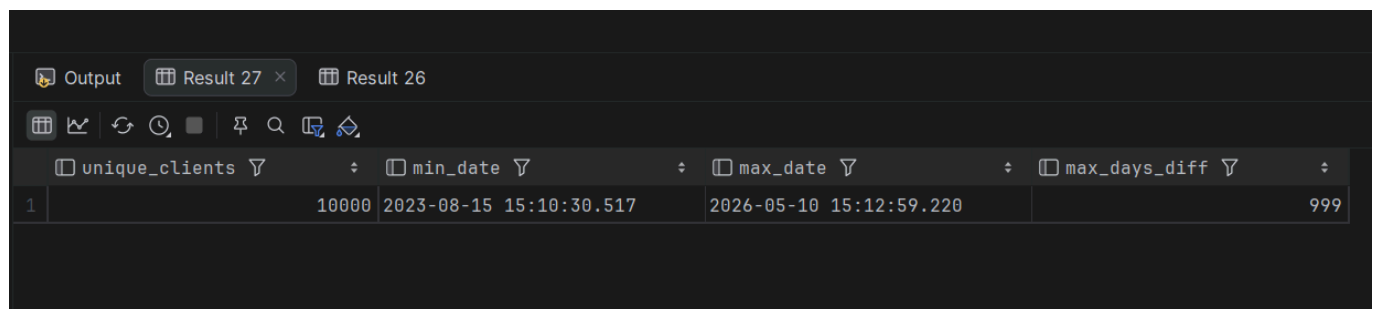
- **OrderID** - unikalny numer zamówienia i klucz główny tabeli
- **CustomerID** - numer identyfikacyjny klienta
- **ProductID** - numer identyfikacyjny produktu
- **OrderDate** - data oraz godzina zamówienia
- **Status** - status zamówienia: completed, pending, cancelled lub error
- **Quantity** - ilość produktów w zamówieniu
- **Price** - cena jednostkowa produktu
- **City** - miasto dostawy

Statystyki

Kategoria	Wartość
Liczba rekordów	500_000
Unikalni klienci	10_000
Unikalne produkty	5_000
Zakres dat [dni]	1_000
Regiony	3
Statusy	3

Kwerenda do wyznaczenia większości statystyk:

```
select count(distinct CustomerID) as unique_clients, min(OrderDate) as min_date,  
       max(OrderDate) as max_date, datediff(day, min(OrderDate), max(OrderDate))  
as max_days_diff from OrdersTest
```



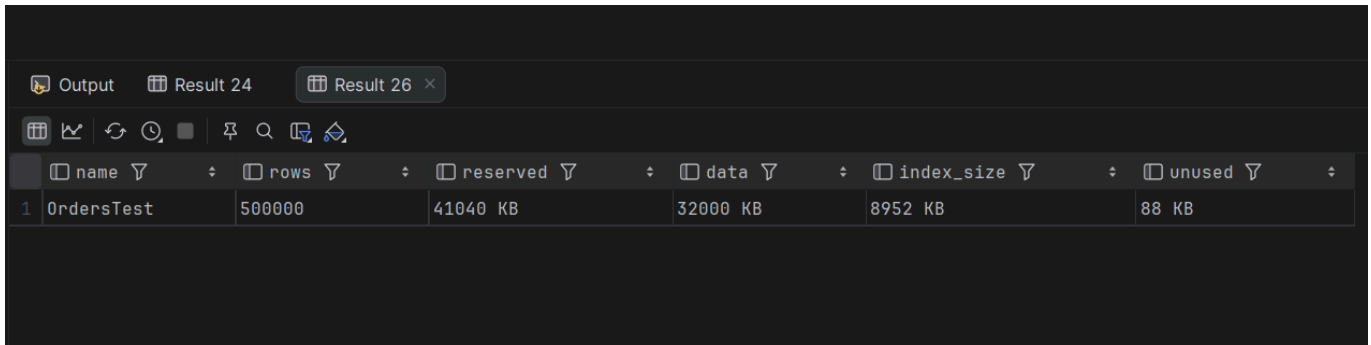
The screenshot shows a SQL Server query results window with the following data:

	unique_clients	min_date	max_date	max_days_diff
1	10000	2023-08-15 15:10:30.517	2026-05-10 15:12:59.220	999

Rozmiar danych

Aby sprawdzić rozmiar miejsca na dysku zajmowanego przez tabelę użyto poniższego polecenia:

```
exec sp_spaceused 'OrdersTest'
```

	name	rows	reserved	data	index_size	unused
1	OrdersTest	500000	41040 KB	32000 KB	8952 KB	88 KB

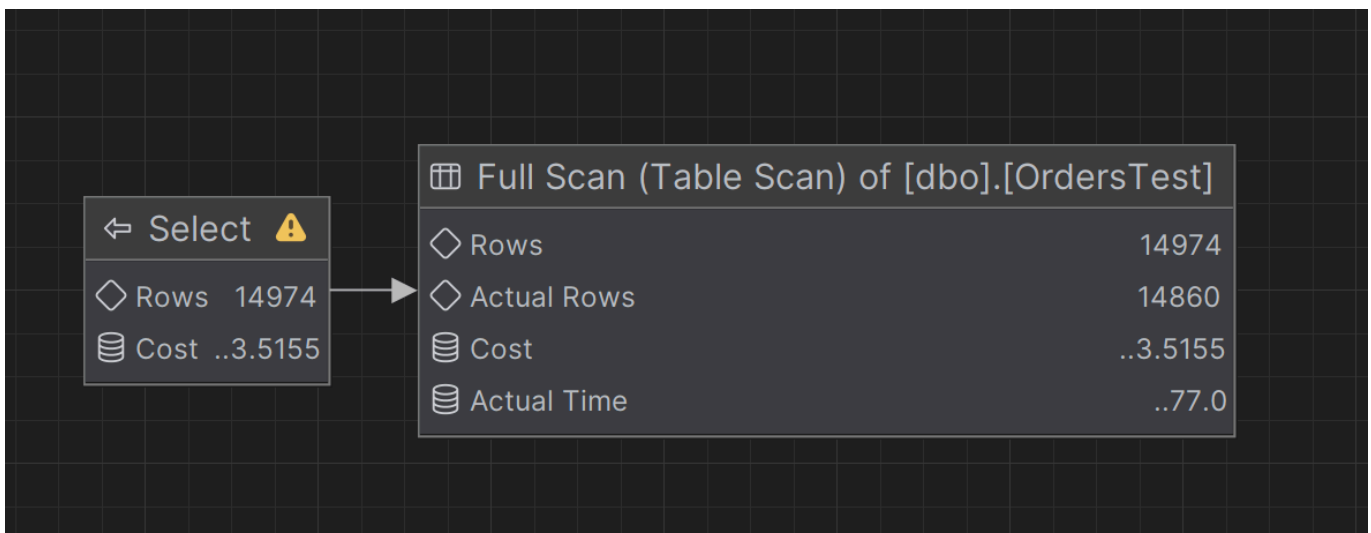
Tabela zajmuje łącznie około 41MB, z czego dane stanowią około 32MB. Póki co w tabeli mamy jeden indeks nieklastrowany na kluczu głównym i zajmuje on niemal 9MB. Wartość unused jest równa zaledwie 88KB, z czego można wnioskować, że strony danych są prawie całkowicie wypełnione.

Indeks klastrowany

Sprawdzimy, jak baza danych radzi sobie z wyszukiwaniem transakcji z konkretnego przedziału czasowego. Chcemy sprawdzić, czy fizyczne posortowanie miliona wierszy według daty skróci czas dostępu do danych.

Zapytanie bez indeksu

```
select * from OrdersTest
where OrderDate between '2026-01-01' and '2026-01-31'
```



Server wykonuje Table Scan czyli musi przejść przez każdy z pół miliona wierszy, aby sprawdzić, czy data pasuje do zakresu. Czas wykonania zapytania wyniósł 157 ms.

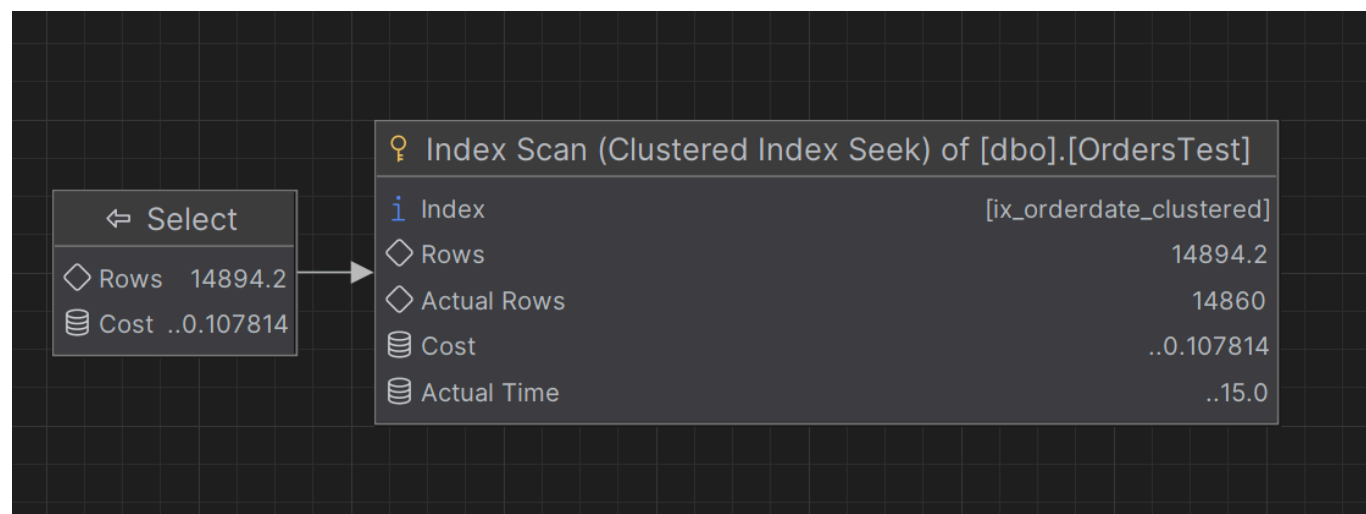
Tworzenie indeksu klastrowanego

Teraz poukładamy tabelę fizycznie według daty. Wybieramy indeks klastrowany, ponieważ w tabeli może być tylko jeden, a data jest najczęstszym kryterium filtrowania + jest nam potrzebna w tym konkretnym przypadku.

```
create clustered index ix_OrderDate_clustered on OrdersTest(OrderDate)
```

Indeks klastrowany przebudowuje tabelę. Od teraz transakcje nie leżą w bazie chaotycznie, ale są poukładane chronologicznie. Dzięki temu baza szukając stycznia wie dokładnie, w którym miejscu na dysku ten styczeń się zaczyna i kończy.

Zapytanie z indeksem



Dzięki zastosowaniu indeksu klastrowanego serwer zamiast Table Scana użył Clustered Index Seeka. Dzięki temu baza nie traci czasu na przeglądanie miliona wierszy, zamiast tego wchodzi bezpośrednio w odpowiednie miejsce na dysku, gdzie zaczynają się dane z wybranego zakresu dat. Czas wykonania zapytania spadł do 35 ms, co jest znaczną poprawą w porównaniu do 95 ms bez indeksu. Różnica jest natomiast wyraźnie widoczna w liczbie odczytywanych stron. Bez indeksu baza danych musiała odczytać 4000 stron (czyli praktycznie wszystkie), aby znaleźć pasujące wiersze, natomiast z indeksem klastrowanym odczytano tylko 9 stron, co świadczy o znacznej poprawie efektywności dostępu do danych.

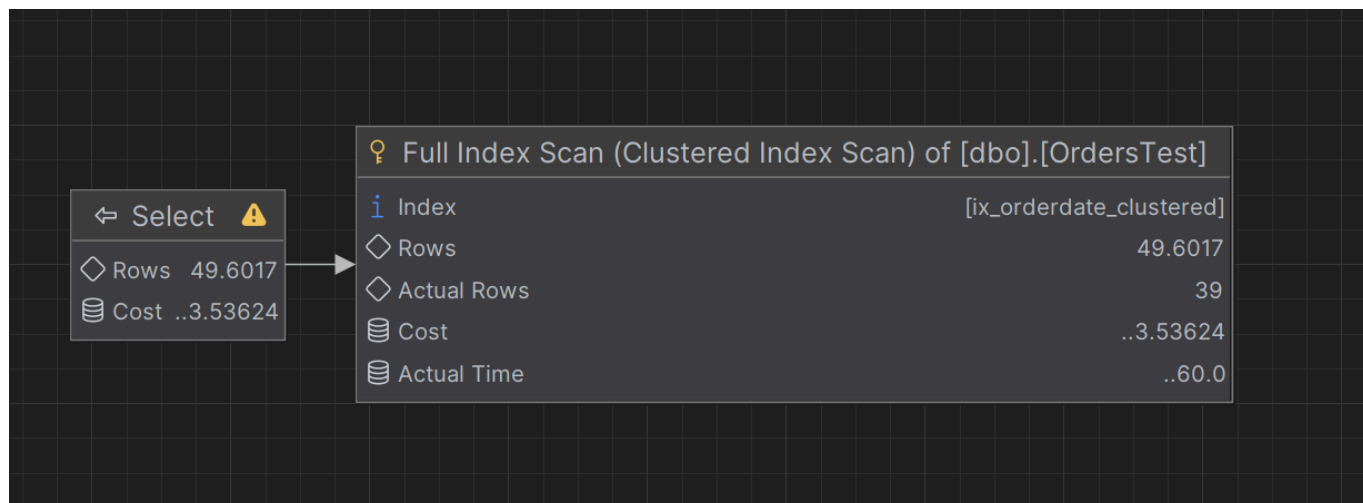
Zapytanie	Koszt	Czas (ms)	Odczytane strony
Bez indeksu	3.5155	95	4000
Z indeksem	0.1078	35	9

Indeks nieklastrowany

Sprawdźmy, jak baza danych radzi sobie z wyszukiwaniem transakcji dla konkretnego klienta. Chcemy sprawdzić czy utworzenie indeksu nieklastrowanego na id klienta przyspieszy dostęp do jego danych w tabeli, która jest już fizycznie posortowana wg daty.

Zapytanie bez indeksu

```
select * from OrdersTest
where CustomerID = 1000
```



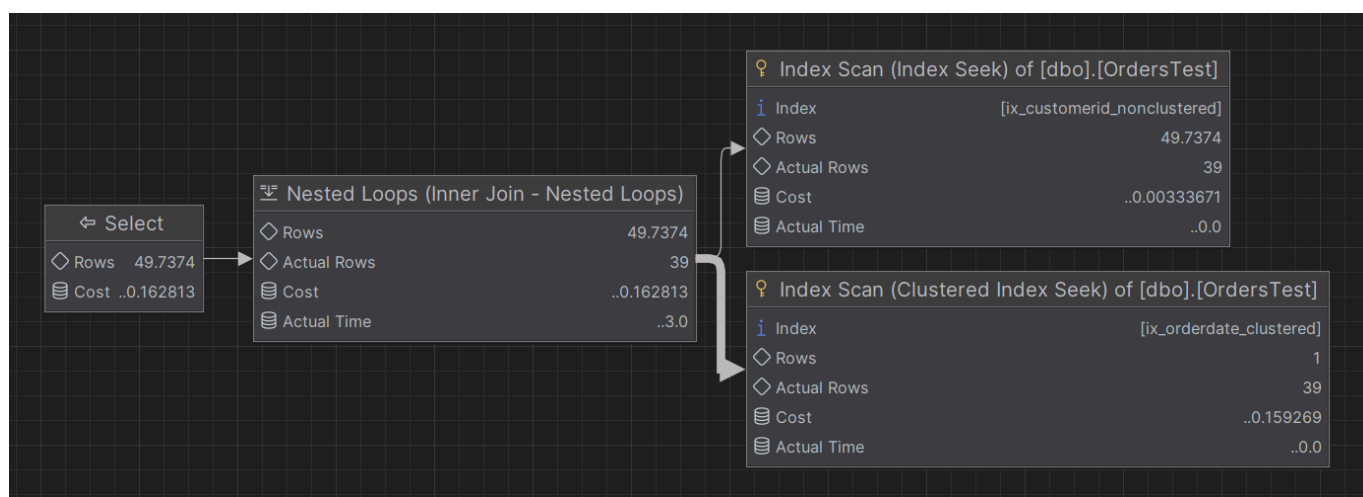
Server wykonuje Clustered Index Scan czyli musi przejść przez każdy z miliona wierszy ułożonych chronologicznie względem OrderDate, aby znaleźć wszystkie transakcje dla wybranego klienta.

Tworzenie indeksu nieklastrowanego

Tworzymy osobną strukturę dla kolumny z id klienta. Wybieramy indeks nieklastrowany, ponieważ nasza tabela ma już swój fizyczny porządek wg daty, a my potrzebujemy szybkiego sposobu na wyszukiwanie po id klienta.

```
create nonclustered index ix_customerid_nonclustered on OrdersTest(CustomerID)
```

Zapytanie z indeksem



Utworzenie indeksu nieklastrowanego całkowicie zmieniło plan wykonania. Baza zrezygnowała z pełnego skanowania za pomocą Clustered Index Scan. Zamiast tego używa operacji Index Seek, aby szybko odnaleźć identyfikator klienta w nowym indeksie, a następnie przez pętlę pobiera brakujące kolumny z tabeli głównej.

Zapytanie	Koszt	Czas (ms)	Odczytane strony
Bez indeksu	3.536	492	4041
Z indeksem	0.163	294	131

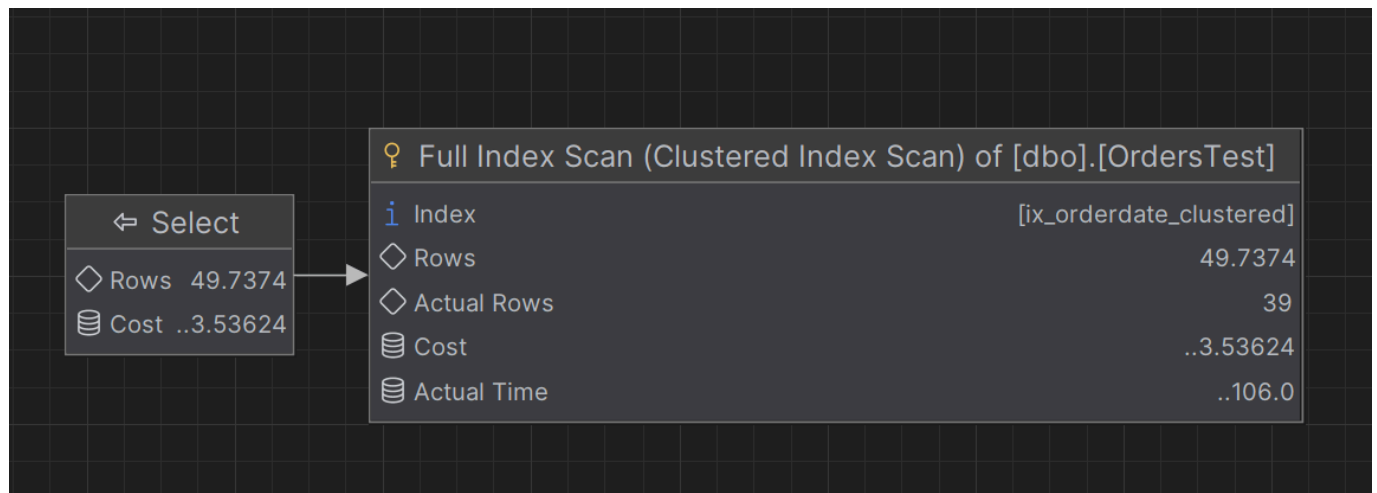
Zastosowanie indeksu nieklastrowanego znacząco poprawiło wydajność zapytania. Liczba odczytów logicznych spadła z 4041 do 131. Koszt zapytania spadł o ponad 90%, czas wykonania również się zmniejszył, lecz mniej znacząco. Optymalizacja zapytań wyszukiwujących pojedyncze wartości przy użyciu indeksu nieklastrowanego jest w tym przypadku skuteczna i chroni bazę przed niepotrzebnym skanowaniem dużej ilości rekordów.

Covering Index

W tym przykładzie będziemy korzystać z utworzonych wyżej indeksów: klastrowanego na OrderDate oraz nieklastrowanego na CustomerID. Sprawdzimy, jak baza danych radzi sobie z zapytaniem, które może być obsłużone wyłącznie przez indeksy, bez konieczności odwoływania się do tabeli głównej.

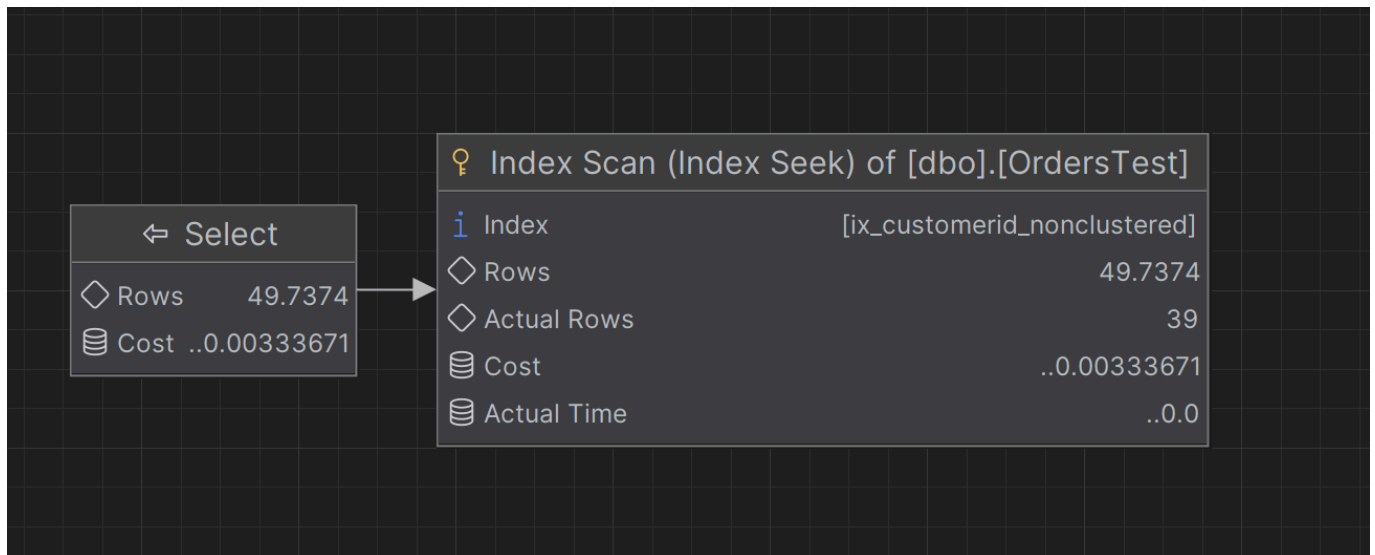
Zapytanie z indeksem pokrywającym

```
select OrderDate from OrdersTest
where CustomerID = 1000
```



Zapytanie bez indeksu (wymuszone użycie klastrowanego po dacie)

```
SELECT OrderDate
FROM OrdersTest WITH (INDEX(ix_OrderDate_clustered))
WHERE CustomerID = 1000;
```



Zapytanie	Koszt	Czas (ms)	Odczytane strony
Bez indeksu	3.536	475	4041
Z indeksem	0.163	412	3

Można zauważyć, że zapytanie z indeksem pokrywającym jest znacznie bardziej efektywne. Baza danych może zaspokoić zapytanie wyłącznie z indeksu nieklastrowanego, ponieważ wszystkie potrzebne kolumny (CustomerID i OrderDate) są zawarte w tym indeksie (ponieważ indeks klastrowany zawiera indeks nieklastrowany). Dzięki temu liczba odczytanych stron spadła do zaledwie 3, a koszt zapytania został drastycznie zredukowany. W przypadku wymuszenia użycia indeksu klastrowanego, baza danych musi nadal skanować dużą liczbę rekordów, co skutkuje znacznie wyższym kosztem i czasem wykonania.

Przykład 3 - Indeks z INCLUDE

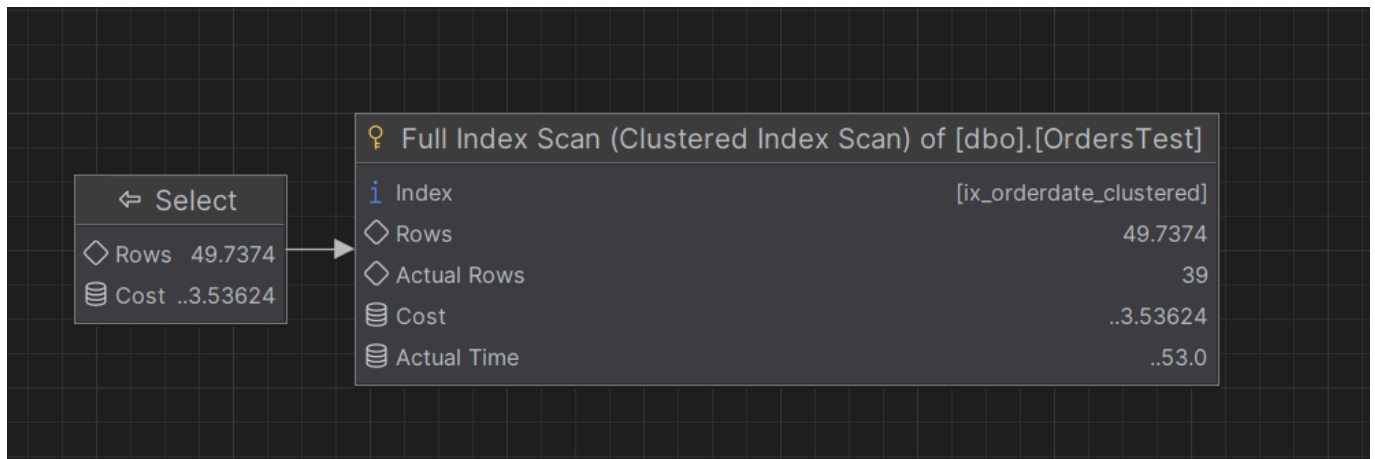
Będziemy chcieli teraz wyświetlić historię klienta, obejmującą datę, kwotę oraz status transakcji.

Indeksy, które na ten moment posiada baza: .png

Na początek sprawdzimy, jak zachowuje się zapytanie bez dodatkowego indeksu:

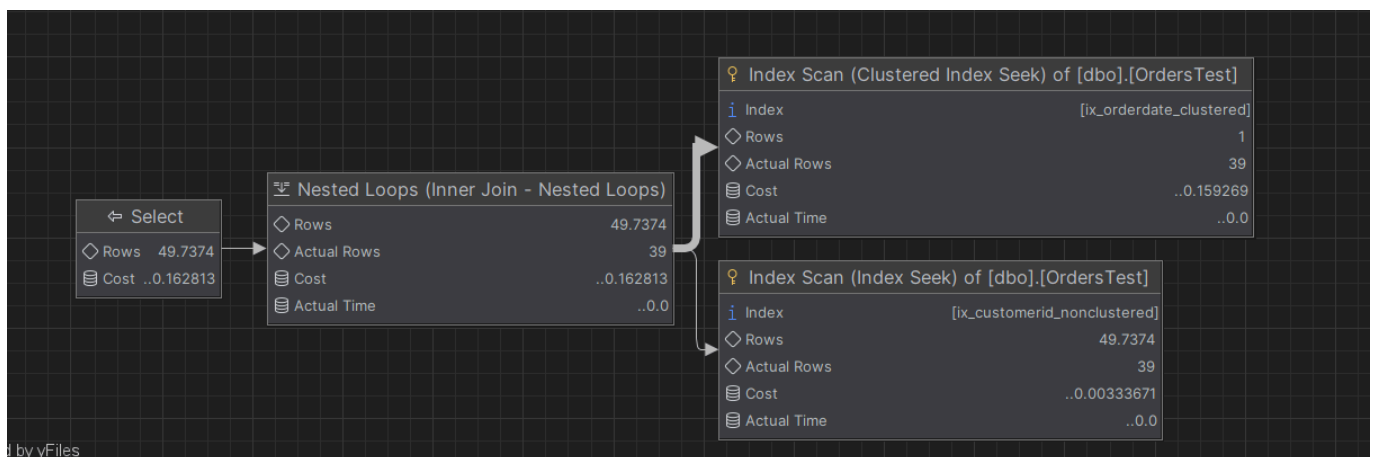
Zapytanie bez indeksu

```
select OrderDate, Status, City from OrdersTest
where customerid = 1234
```



Zapytanie z indeksem nieklastrowanym na customerid

```
select OrderDate, Status, City from OrdersTest
where customerid = 1000
```

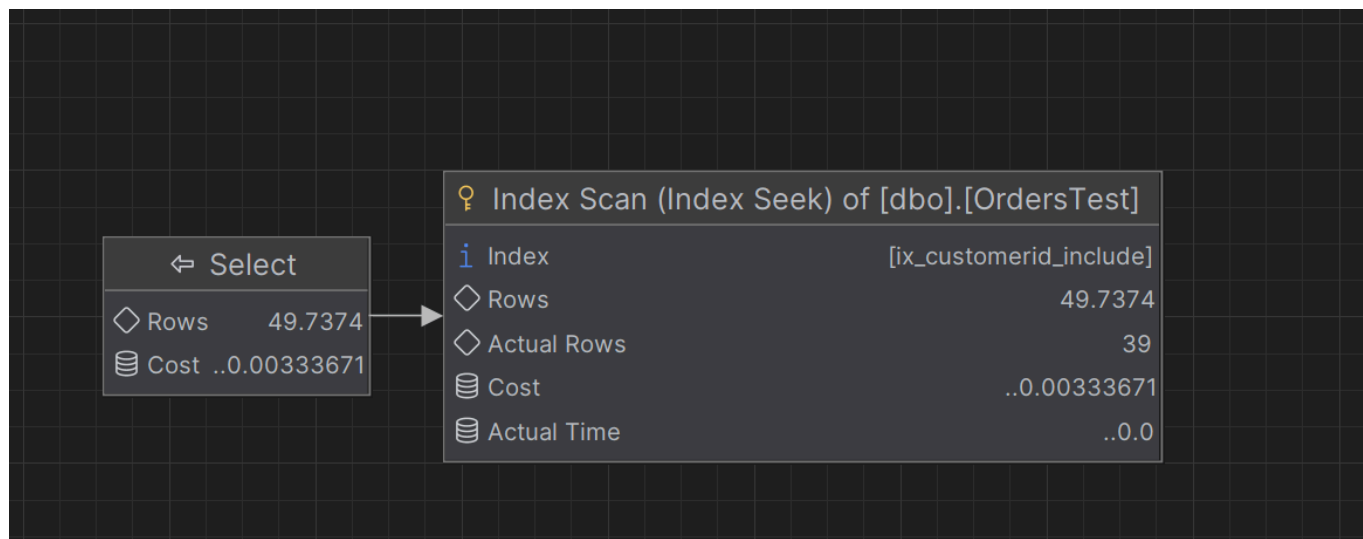


Zapytanie z indeksem nieklastrowanym na customerid, który zawiera dodatkowe kolumny (INCLUDE)

Tworzenie indeksu z INCLUDE

```
create nonclustered index ix_customerid_include
on OrdersTest(CustomerID) include (Status, City)
```

Resultat zapytania z indeksem z INCLUDE



Zapytanie	Koszt	Czas (ms)	Odczytane strony
Bez indeksu	3.536	460	4041
Z indeksem	0.162	392	131
Include	0.003	365	3

Zastosowanie indeksu z INCLUDE pozwoliło na jeszcze większą optymalizację zapytania. Baza danych była w stanie zaspokoić zapytanie wyłącznie z indeksu, ponieważ wszystkie potrzebne kolumny (CustomerID, Status, City i OrderDate będące kluczem indeksu klastrowanego) były zawarte w tym indeksie. Dzięki temu liczba odczytanych stron spadła do zaledwie 3, a koszt zapytania został drastycznie zredukowany w porównaniu do zapytania bez indeksu oraz z indeksem nieklastrowanym bez INCLUDE. Czasowo też różnica jest widoczna, jednak nie aż tak, ze względu na to, że we wszystkich przypadkach do czasu był wliczany czas komunikacji z bazą danych.

Filtered Index (Indeks warunkowy)

W tym przykładzie sprawdzimy, jak zoptymalizować zapytania o rzadkie zdarzenia. Zamiast indeksować całą tabelę, zostanie stworzony indeks zawierający tylko rekordy ze statusem zamówienia jako anulowane. Dzięki temu zapytania szukające błędów będą znacznie szybsze, ponieważ będą operować na znacznie mniejszym zbiorze danych.

Sprawdzenie liczby anulowanych zamówień

```
select count(*) from OrdersTest
where status = 'Cancelled'
```

Zmniejszenie liczby anulowanych zamówień do 5 procent

```
DECLARE @TotalRows INT;
DECLARE @TargetCancelled INT;
DECLARE @CurrentCancelled INT;
```

```
DECLARE @RowsToUpdate INT;

SELECT @TotalRows = COUNT(*)
FROM OrdersTest;

SET @TargetCancelled = @TotalRows * 0.05;

SELECT @CurrentCancelled = COUNT(*)
FROM OrdersTest
WHERE Status = 'Cancelled';

SET @RowsToUpdate = @CurrentCancelled - @TargetCancelled;

UPDATE TOP (@RowsToUpdate) OrdersTest
SET Status = 'Pending'
WHERE Status = 'Cancelled';
```

Ilość zamówień per status

```
select Status, count(*) as num from OrdersTest
group by Status
```

Session | Result 57 | count(*):int

	Status	num
1	Cancelled	25000
2	Completed	400000
3	Pending	75000

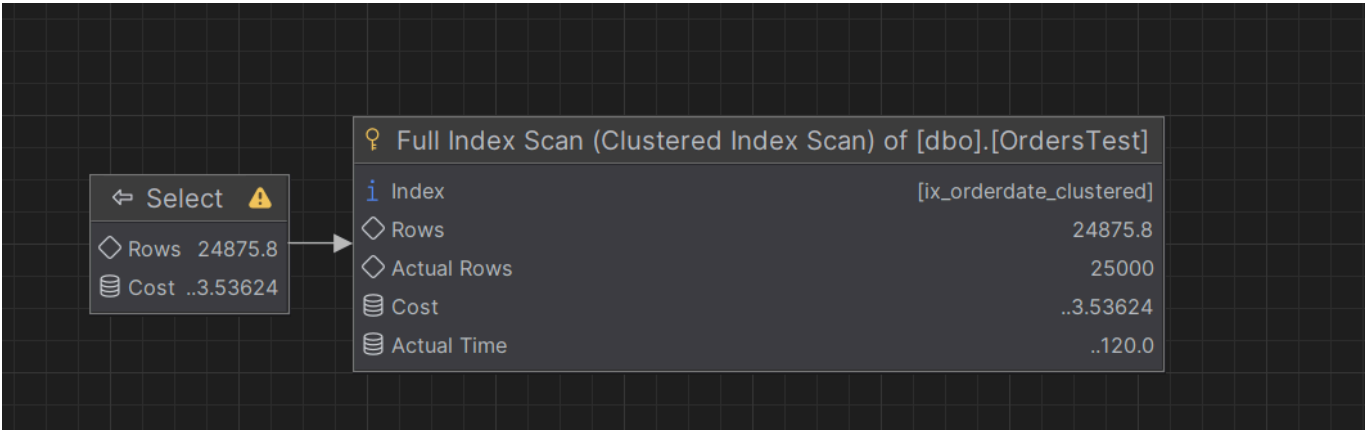
3 rows

Zapytanie

Szukanie anulowanych zamówień, wraz z informacją o kliencie i mieście dostawy.

```
select OrderID, CustomerID, City
from OrdersTest
where status = 'Cancelled'
```

Bez dedykowanego indeksu

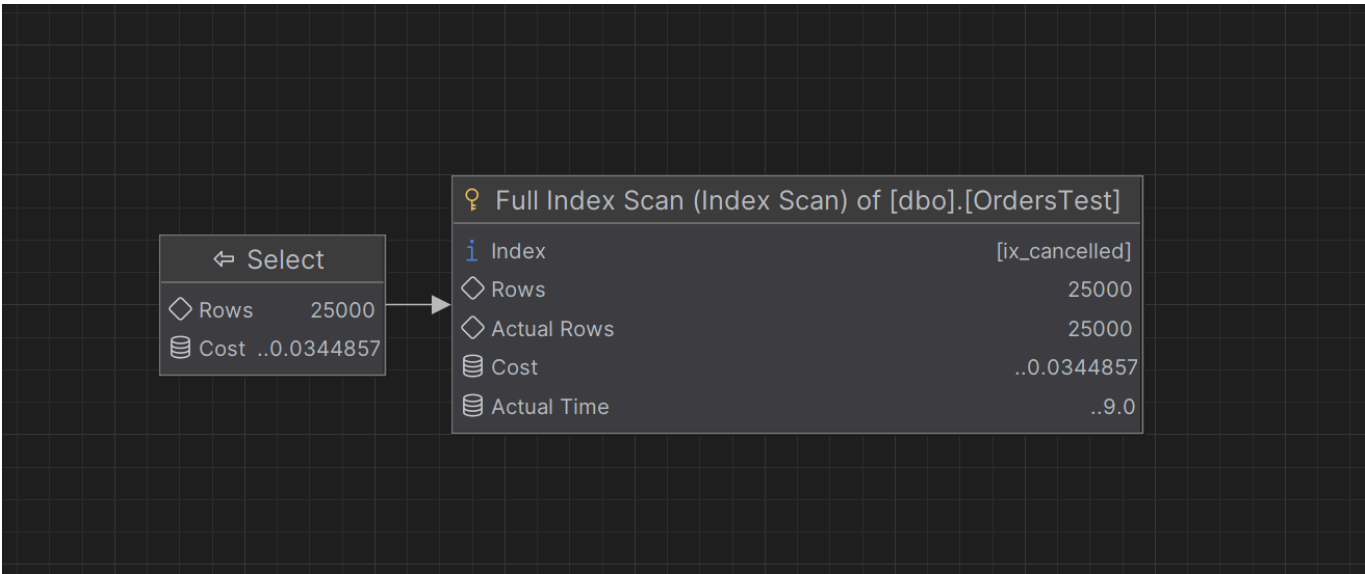


Silnik bazy danych wykonuje pełne skanowanie głównej tabeli za pomocą Clustered Index Scan.

Tworzenie indeksu filtrowanego

```
create nonclustered index ix_cancelled
on OrdersTest (OrderID)
include (CustomerID, City)
where Status = 'Cancelled'
```

Z Indekssem filtrowany



Plan zapytania uprościł się. Baza wykonała operację Index Scan na indeksie filtrowanym. Wydaje się to być w tym przypadku optymalne, ponieważ indeks zawiera wyłącznie szukane błędy, więc silnik odczytuje po kolei całą jego zawartość bez konieczności nawigowania po drzewie indeksu jak byloby w przypadku Index Seeka.

Zapytanie	Koszt	Czas (ms)	Odczytane strony
Brak indeksu	3.536	15	80
Z indeksem	0.034	6	5

Jak widać indeksy warunkowe dla danych o wysokiej selektywności bardzo dobrze sobie radzą. Liczba odczytanych stron spadła z ponad 80 do 5, a koszt zapytania zmniejszył o około 98 %.

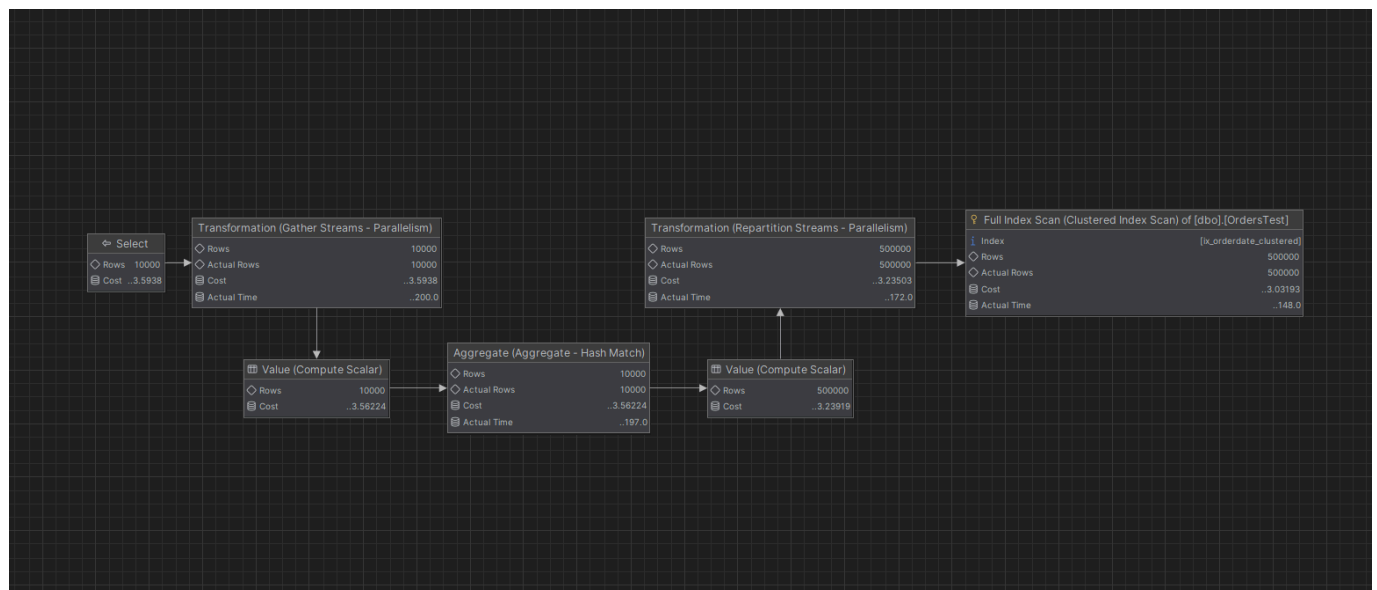
Indeks kolumnowy

W tym przykładzie zostanie zanalizowane zapytanie wymagające pogrupowania i zagregowania dużej liczby rekordów dla poszczególnych klientów.

Zapytanie

```
select
    customerID,
    count(*) as orders_count,
    sum(Quantity * Price) as total_bought
from OrdersTest
group by customerID
```

Bez indeksu

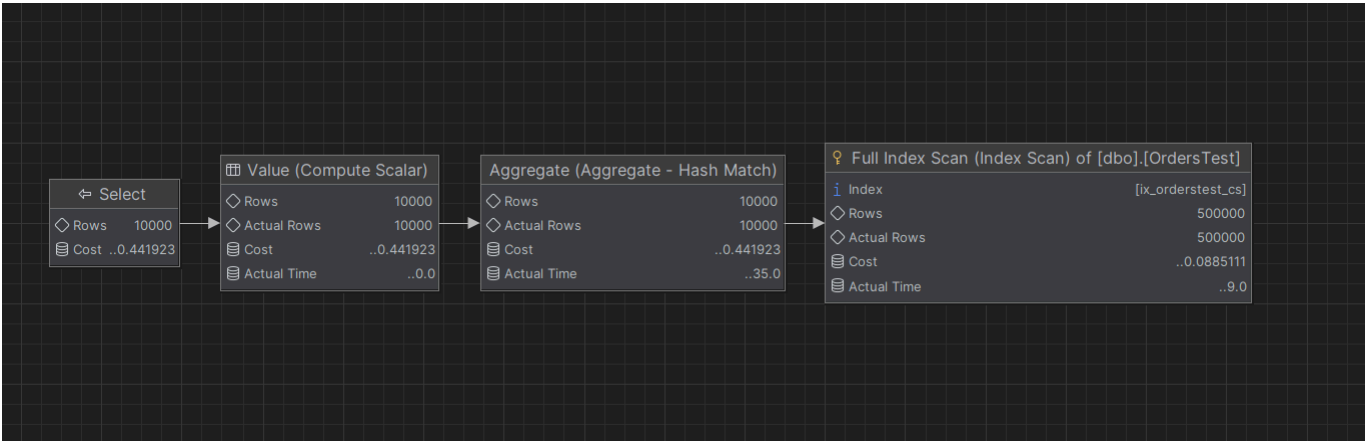


Silnik bazy danych użył indeksu klastrowanego, aby odczytać dane, ale musiał przetworzyć wszystkie rekordy, aby pogrupować je według klienta i obliczyć sumę. Plan zapytania zawiera operacje zrównoleglenia i przesyłania strumieni, co wskazuje na to, że baza danych próbuje zoptymalizować przetwarzanie dużej ilości danych, ale nadal musi odczytać wszystkie rekordy, co jest kosztowne.

Tworzenie indeksu kolumnowego

```
CREATE NONCLUSTERED COLUMNSTORE INDEX IX_OrdersTest_CS
ON OrdersTest(CustomerID, Quantity, Price);
```

Z indeksem



Plan zapytania uległ znacznemu uproszczeniu. Silnik wykonuje operację Index Scan na nowym indeksie kolumnowym. Operatory zrównoleglenia i przesyłania strumieni zostały wyeliminowane z planu, a proces agregacji jest realizowany jednoetapowo.

Zapytanie	Koszt	Czas (ms)	Odczytane strony
Bez indeksu	3.592	209	4239
Z indeksem	0.442	91	1305 (lob logical reads)

Indeks kolumnowy znacząco poprawił wydajność zapytania agregującego dużą ilość danych. Liczba odczytanych stron spadła z 4239 do 1305, a koszt zapytania został zredukowany o ponad 87%. Czas wykonania również uległ znacznemu skróceniu, co świadczy o efektywności indeksów kolumnowych w przypadku zapytań analitycznych i agregujących.

zadanie	pkt
1	6
2	2
3	5 (3 pkt. za eksperymenty + 2 dodatkowe za ciekawe/oryginalne przykłady)
razem	13